5.1 Background

Introduction

Overview

A system typically consists of several (perhaps hundreds or even thousands) of threads running either concurrently or in parallel. Threads often share user data. Meanwhile, the operating system continuously updates various data structures to support multiple threads. A *race condition* exists when access to shared data is not controlled, possibly resulting in corrupt data values.

Process synchronization involves using tools that control access to shared data to avoid race conditions. These tools must be used carefully, as their incorrect use can result in poor system performance, including deadlock.

A ***cooperating process*** is one that can affect or be affected by other processes executing in the system. Cooperating processes can either directly share a logical address space (that is, both code and data) or be allowed to share data only through shared memory or message passing. Concurrent access to shared data may result in data inconsistency, however. In this chapter, we discuss various mechanisms to ensure the orderly execution of cooperating processes that share a logical address space, so that data consistency is maintained.

Chapter objectives

- Describe the critical-section problem and illustrate a race condition.
- Illustrate hardware solutions to the critical-section problem using memory barriers, compare-and-swap operations, and atomic variables.
- Demonstrate how mutex locks, semaphores, monitors, and condition variables can be used to solve the critical-section problem.
- Evaluate tools that solve the critical-section problem in low-, moderate-, and high-contention scenarios.

Section glossary

cooperating process: A process that can affect or be affected by other processes executing in the system.

Background

We've already seen that processes can execute concurrently or in parallel. Section CPU scheduling introduced the role of process scheduling and described how the CPU scheduler switches rapidly between processes to provide concurrent execution. This means that one process may only partially complete execution before another process is scheduled. In fact, a process may be interrupted at any point in its instruction stream, and the processing core may be assigned to execute instructions of another process. Additionally, Section 3.2 introduced parallel execution, in which two instruction streams (representing different processes) execute simultaneously on separate processing cores. In this chapter, we explain how concurrent or parallel execution can contribute to issues involving the integrity of data shared by several processes.

Let's consider an example of how this can happen. In the chapter Processes, we described how a bounded buffer could be used to enable processes to share memory.

We now return to our consideration of the bounded buffer. As we pointed out, our original solution allowed at most `BUFFER_SIZE - 1` items in the buffer at the same time. Suppose we want to modify the algorithm to remedy this deficiency. One possibility is to add an integer variable, `count`, initialized to 0. `count` is incremented every time we add a new item to the buffer and is decremented every time we remove one item from the buffer. The code for the producer process can be modified as follows:

```
while (true) {
     /* produce an item in next_produced */

     while (count ==  BUFFER_SIZE)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
     count++;
}
```

The code for the consumer process can be modified as follows:

```
while (true) {
     while (count == 0)
        ; /* do nothing */

     next_consumed = buffer[out];
     out = (out + 1) % BUFFER_SIZE;
     count--;
```

```
    /* consume the item in next_consumed */
}
```

Although the producer and consumer routines shown above are correct separately, they may not function correctly when executed concurrently. As an illustration, suppose that the value of the variable **count** is currently 5 and that the producer and consumer processes concurrently execute the statements "**count++**" and "**count--**". Following the execution of these two statements, the value of the variable **count** may be 4, 5, or 6! The only correct result, though, is **count** == 5, which is generated correctly if the producer and consumer execute separately.

We can show that the value of **count** may be incorrect as follows. Note that the statement "**count++**" may be implemented in machine language (on a typical machine) as follows:

```
register1 = count
register1 = register1 + 1
count = register1
```

where *register*$_1$ is one of the local CPU registers. Similarly, the statement "count--" is implemented as follows:

```
register2 = count
register2 = register2 - 1
count = register2
```

where again *register*$_2$ is one of the local CPU registers. Even though *register*$_1$ and *register*$_2$ may be the same physical register, remember that the contents of this register will be saved and restored by the interrupt handler (Section I/O Structure).

The concurrent execution of "**count++**" and "**count--**" is equivalent to a sequential execution in which the lower-level statements presented previously are interleaved in some arbitrary order (but the order within each high-level statement is preserved). One such interleaving is the following:

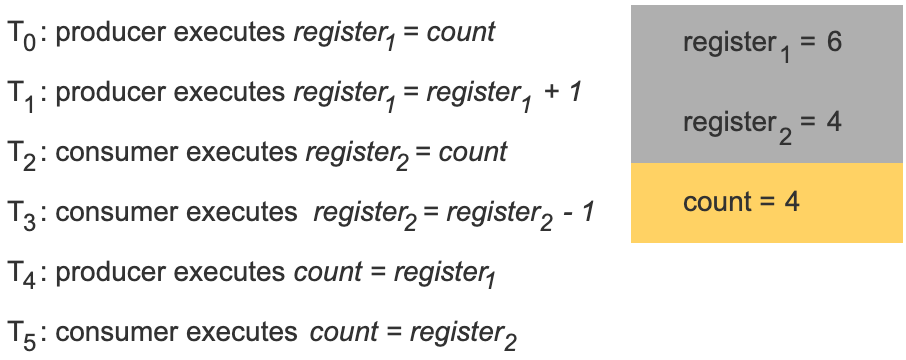
```
T0: producer execute register1 = count          {register1 = 5}
T1: producer execute register1 = register1 + 1 {register1 = 6}
T2: consumer execute register2 = count          {register2 = 5}
T3: consumer execute register2 = register2 -1  {register2 = 4}
T4: producer execute count = register1          {count = 6}
T5: consumer execute count = register2          {count = 4}
```

Notice that we have arrived at the incorrect state "**count** == 4", indicating that four buffers are full, when, in fact, five buffers are full. If we reversed the order of the statements at T_4 and T_5, we would arrive at the incorrect state "**count** == 6".

We would arrive at this incorrect state because we allowed both processes to manipulate the variable **count** concurrently. A situation like this, where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes

place, is called a ***race condition***. To guard against the race condition above, we need to ensure that only one process at a time can be manipulating the variable `count`. To make such a guarantee, we require that the processes be synchronized in some way.

PARTICIPATION ACTIVITY 5.1.1: Race Condition.



Animation content:

Static figure: A series of statements are ordered from time T_0 to time T_5. These statements manipulate the contents of the CPU registers register_1 and register_2. At time T_0: "producer executes register_1 = count". At time T_1: "producer executes register_1 = register_1 + 1". At time T_2: "consumer executes register_2 = count". At time T_3: "producer executes register_2 = register_2 - 1". At time T_4: "producer executes count = register_1". At time T_5: "consumer executes count = register_2". Register_1's value is 6. Register_2's value is 4. Count's value is 4.

Animation captions:

1. The producer sets register1 to the current value of count which is 5.
2. Register1 is incremented by 1.
3. The consumer sets register2 to the current value of count which is 5.
4. Register2 is decremented by 1.
5. The producer sets count to the value of register1 which is 6.
6. The consumer sets count to the value of register2 which is 4 which is incorrect as count should be 5.

Situations such as the one just described occur frequently in operating systems as different parts of the system manipulate resources. Furthermore, as we have emphasized in earlier chapters, the prominence of multicore systems has brought an increased emphasis on developing multithreaded applications. In such applications, several threads—which are quite possibly sharing data—are running in parallel on different processing cores. Clearly, we want any changes that result from such activities not to interfere with one another. Because of the importance of this issue, we devote a major portion of this chapter to ***process synchronization*** and ***coordination*** among cooperating processes.

PARTICIPATION ACTIVITY 5.1.2: Section review questions.

A race condition can occur ____.

- when several threads read the same data concurrently
- when several threads try to access and modify the same data concurrently
- only if the outcome of execution does not depend on the order in which instructions are executed

Section glossary

race condition: A situation in which two threads are concurrently trying to change the value of a variable.

process synchronization: Coordination of access to data by two or more threads or processes.

coordination: Ordering of the access to data by multiple threads or processes.

5.2 The critical-section problem

We begin our consideration of process synchronization by discussing the so-called critical-section problem. Consider a system consisting of n processes $\{P_0, P_1, \ldots, P_{n-1}\}$. Each process has a segment of code, called a ***critical section***, in which the process may be accessing — and updating — data that is shared with at least one other process. The important feature of the system is that, when one process is executing in its critical section, no other process is allowed to execute in its critical section. That is, no two

processes are executing in their critical sections at the same time. The ***critical-section problem*** is to design a protocol that the processes can use to synchronize their activity so as to cooperatively share data. Each process must request permission to enter its critical section. The section of code implementing this request is the ***entry section***. The critical section may be followed by an ***exit section***. The remaining code is the ***remainder section***. The general structure of a typical process is shown in Figure 5.2.1. The entry section and exit section are shown to highlight these important segments of code.

Figure 5.2.1: General structure of a typical process.

```
while (true) {

    entry section

        critical section

    exit section

        remainder section

}
```

A solution to the critical-section problem must satisfy the following three requirements:

1. **Mutual exclusion.** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress.** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.
3. **Bounded waiting.** There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

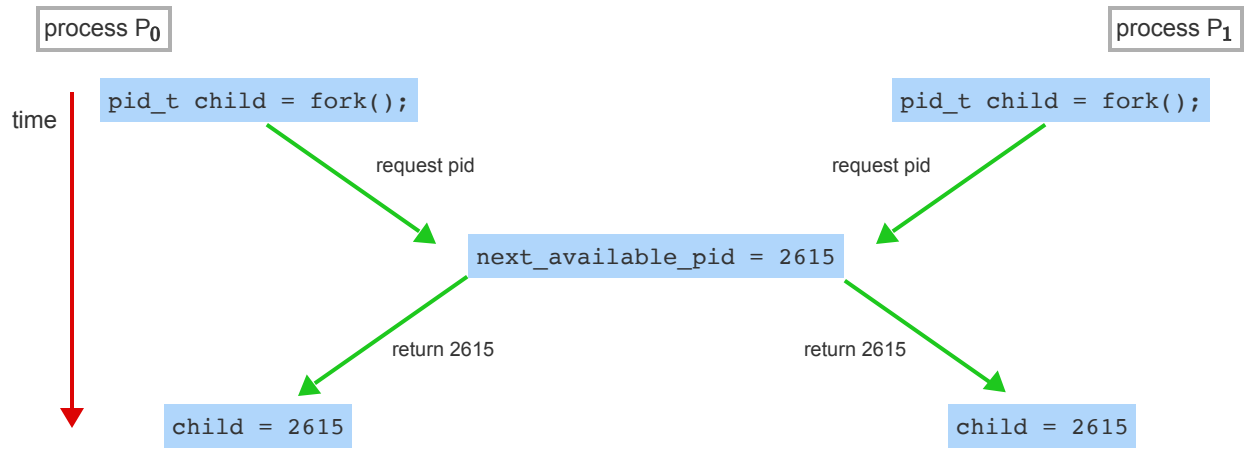
We assume that each process is executing at a nonzero speed. However, we can make no assumption concerning the relative speed of the n processes.

At a given point in time, many kernel-mode processes may be active in the operating system. As a result, the code implementing an operating system (***kernel code***) is subject to several possible race conditions. Consider as an example a kernel data structure that maintains a list of all open files in the system. This list must be modified when a new file is opened or closed (adding the file to the list or removing it from the list). If two processes were to open files simultaneously, the separate updates to this list could result in a race condition.

Another example is illustrated in the animation below. In this situation, two processes, P_0 and P_1, are creating child processes using the `fork()` system call. Recall from Section Process Creation that

`fork()` returns the process identifier of the newly created process to the parent process. In this example, there is a race condition on the variable kernel variable `next_available_pid` which represents the value of the next available process identifier. Unless mutual exclusion is provided, it is possible the same process identifier number could be assigned to two separate processes.

PARTICIPATION ACTIVITY 5.2.1: Race condition when assigning a pid.



Animation content:

Step 1: Process P0 and P1 are creating child processes using the fork() system call. Step 2: There is a race condition on the kernel variable next_available_pid which represents the value of the next available process identifier Step 3: Unless mutual exclusion is provided, it is possible the same process identifer number could be assigned to two separate processes

Animation captions:

1. Process P_0 and P_1 are creating child processes using the fork() system call.
2. There is a race condition on the kernel variable next_available_pid which represents the value of the next available process identifier.
3. Unless mutual exclusion is provided, it is possible the same process identifier number could be assigned to two separate processes.

Other kernel data structures that are prone to possible race conditions include structures for maintaining memory allocation, for maintaining process lists, and for interrupt handling. It is up to kernel developers to

ensure that the operating system is free from such race conditions.

The critical-section problem could be solved simply in a single-core environment if we could prevent interrupts from occurring while a shared variable was being modified. In this way, we could be sure that the current sequence of instructions would be allowed to execute in order without preemption. No other instructions would be run, so no unexpected modifications could be made to the shared variable.

Unfortunately, this solution is not as feasible in a multiprocessor environment. Disabling interrupts on a multiprocessor can be time consuming, since the message is passed to all the processors. This message passing delays entry into each critical section, and system efficiency decreases. Also consider the effect on a system's clock if the clock is kept updated by interrupts.

Two general approaches are used to handle critical sections in operating systems: ***preemptive kernels*** and ***nonpreemptive kernels***. A preemptive kernel allows a process to be preempted while it is running in kernel mode. A nonpreemptive kernel does not allow a process running in kernel mode to be preempted; a kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU.

Obviously, a nonpreemptive kernel is essentially free from race conditions on kernel data structures, as only one process is active in the kernel at a time. We cannot say the same about preemptive kernels, so they must be carefully designed to ensure that shared kernel data are free from race conditions. Preemptive kernels are especially difficult to design for SMP architectures, since in these environments it is possible for two kernel-mode processes to run simultaneously on different CPU cores.

Why, then, would anyone favor a preemptive kernel over a nonpreemptive one? A preemptive kernel may be more responsive, since there is less risk that a kernel-mode process will run for an arbitrarily long period before relinquishing the processor to waiting processes. (Of course, this risk can also be minimized by designing kernel code that does not behave in this way.) Furthermore, a preemptive kernel is more suitable for real-time programming, as it will allow a real-time process to preempt a process currently running in the kernel.

PARTICIPATION ACTIVITY 5.2.2: Section review questions.

1) A(n) _____ refers to the section of the code which is accessing data that may be modified by another process executing concurrently.
   - ○ critical section
   - ○ entry section
   - ○ remainder section

2) _____ is the requirement that if a process is in its critical section, no other processes can be in their critical sections at the same time.

- Progress
- Bounded waiting
- Mutual exclusion

3) Which of the following types of operating system kernel is free from race conditions?

- Nonpreemptive kernel
- Preemptive kernel
- No operating system kernel is free from race conditions

Section glossary

critical section: A section of code responsible for changing data that must only be executed by one thread or process at a time to avoid a race condition.

entry section: The section of code within a process that requests permission to enter its critical section.

exit section: The section of code within a process that cleanly exits the critical section.

remainder section: Whatever code remains to be processed after the critical and exit sections.

preemptive kernel: A type of kernel that allows a process to be preempted while it is running in kernel mode.

nonpreemptive kernels: A type of kernel that does not allow a process running in kernel mode to be preempted; a kernel-mode process will run until it exits kernel mode, blocks, or voluntarily yields control of the CPU.

5.3 Peterson's solution

Next, we illustrate a classic software-based solution to the critical-section problem known as ***Peterson's solution***. Because of the way modern computer architectures perform basic machine-language

instructions, such as `load` and `store`, there are no guarantees that Peterson's solution will work correctly on such architectures. However, we present the solution because it provides a good algorithmic description of solving the critical-section problem and illustrates some of the complexities involved in designing software that addresses the requirements of mutual exclusion, progress, and bounded waiting.

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections. The processes are numbered P_0 and P_1. For convenience, when presenting P_i, we use P_j to denote the other process; that is, `j` equals `i - 1`.

Peterson's solution requires the two processes to share two data items:

```
int turn;
boolean flag[2];
```

The variable `turn` indicates whose turn it is to enter its critical section. That is, if `turn == i`, then process P_i is allowed to execute in its critical section. The `flag` array is used to indicate if a process is ready to enter its critical section. For example, if `flag[i]` is `true`, P_i is ready to enter its critical section. With an explanation of these data structures complete, we are now ready to describe the algorithm shown in Figure 5.3.1.

Figure 5.3.1: The structure of process P_i in Peterson's solution.

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ;

        /* critical section */

    flag[i] = false;

        /*remainder section */
}
```

To enter the critical section, process P_i first sets `flag[i]` to be `true` and then sets `turn` to the value `j`, thereby asserting that if the other process wishes to enter the critical section, it can do so. If both processes try to enter at the same time, `turn` will be set to both `i` and `j` at roughly the same time. Only one of these assignments will last; the other will occur but will be overwritten immediately. The eventual value of `turn` determines which of the two processes is allowed to enter its critical section first.

We now prove that this solution is correct. We need to show that:

1. Mutual exclusion is preserved.
2. The progress requirement is satisfied.
3. The bounded-waiting requirement is met.

To prove property 1, we note that each P_i enters its critical section only if either `flag[j]` == `false` or `turn` == `i`. Also note that, if both processes can be executing in their critical sections at the same time, then `flag[0]` == `flag[1]` == `true`. These two observations imply that P_0 and P_1 could not have successfully executed their `while` statements at about the same time, since the value of `turn` can be either 0 or 1 but cannot be both. Hence, one of the processes—say, P_j—must have successfully executed the `while` statement, whereas P_i had to execute at least one additional statement ("`turn` == `j`"). However, at that time, `flag[j]` == `true` and `turn` == `j`, and this condition will persist as long as P_j is in its critical section; as a result, mutual exclusion is preserved.

To prove properties 2 and 3, we note that a process P_i can be prevented from entering the critical section only if it is stuck in the `while` loop with the condition `flag[j]` == `true` and `turn` == `j`; this loop is the only one possible. If P_j is not ready to enter the critical section, then `flag[j]` == `false`, and P_i can enter its critical section. If P_j has set `flag[j]` to `true` and is also executing in its `while` statement, then either `turn` == `i` or `turn` == `j`. If `turn` == `i`, thenP_i will enter the critical section. If `turn` == `j`, then P_j will enter the critical section. However, once P_j exits its critical section, it will reset `flag[j]` to `false`, allowing P_i to enter its critical section. If P_j resets `flag[j]` to `true`, it must also set `turn` to `i`. Thus, since P_i does not change the value of the variable `turn` while executing the `while` statement, P_i will enter the critical section (progress) after at most one entry by P_j (bounded waiting).

As mentioned at the beginning of this section, Peterson's solution is not guaranteed to work on modern computer architectures for the primary reason that, to improve system performance, processors and/or compilers may reorder read and write operations that have no dependencies. For a single-threaded application, this reordering is immaterial as far as program correctness is concerned, as the final values are consistent with what is expected. (This is similar to balancing a checkbook—the actual order in which credit and debit operations are performed is unimportant, because the final balance will still be the same.) But for a multithreaded application with shared data, the reordering of instructions may render inconsistent or unexpected results.

As an example, consider the following data that are shared between two threads:

```
boolean flag = false;
int x = 0;
```

where Thread 1 performs the statements

```
while (!flag)
   ;
print x;
```

and Thread 2 performs

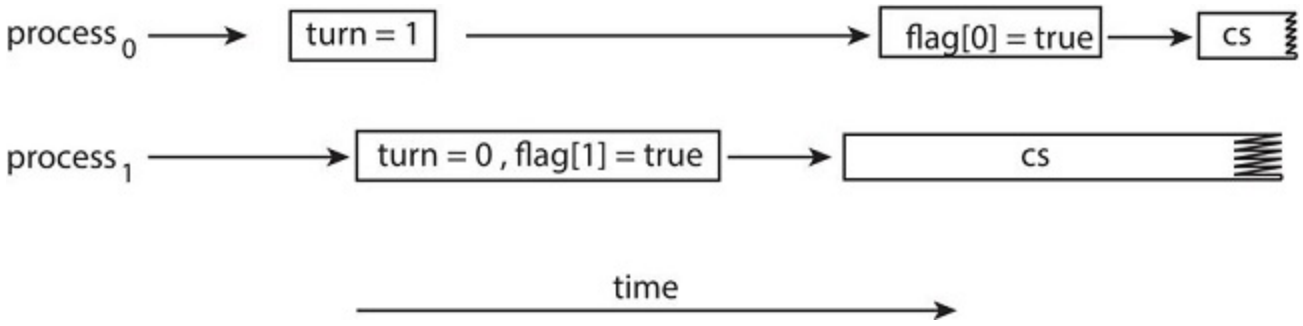
```
x = 100;
flag = true;
```


The expected behavior is, of course, that Thread 1 outputs the value 100 for variable `x`. However, as there are no data dependencies between the variables `flag` and `x`, it is possible that a processor may reorder the instructions for Thread 2 so that `flag` is assigned `true` before assignment of `x = 100`. In this situation, it is possible that Thread 1 would output 0 for variable `x`. Less obvious is that the processor may

also reorder the statements issued by Thread 1 and load the variable `x` before loading the value of `flag`. If this were to occur, Thread 1 would output 0 for variable `x` even if the instructions issued by Thread 2 were not reordered.

How does this affect Peterson's solution? Consider what happens if the assignments of the first two statements that appear in the entry section of Peterson's solution in Figure 5.3.1 are reordered; it is possible that both threads may be active in their critical sections at the same time, as shown in Figure 5.3.2.

Figure 5.3.2: The effects of instruction reordering in Peterson's solution.



As you will see in the following sections, the only way to preserve mutual exclusion is by using proper synchronization tools. Our discussion of these tools begins with primitive support in hardware and proceeds through abstract, high-level, software-based APIs available to both kernel developers and application programmers.

PARTICIPATION ACTIVITY 5.3.1: Section review questions.

1) In Peterson's solution, the ____ variable indicates if process i is ready to enter its critical section.
   - `turn`
   - `lock`
   - `flag[i]`

2) What best explains why Peterson's solution does not solve the critical section problem on modern computer architectures?

- Modern computer architectures may reorder instructions.
- Peterson's solution does not preserve the bounded waiting requirement.

Section glossary

Peterson's solution: A historically interesting algorithm for implementing critical sections.

5.4 Hardware support for synchronization

We have just described one software-based solution to the critical-section problem. (We refer to it as a ***software-based*** solution because the algorithm involves no special support from the operating system or specific hardware instructions to ensure mutual exclusion.) However, as discussed, software-based solutions are not guaranteed to work on modern computer architectures. In this section, we present three hardware instructions that provide support for solving the critical-section problem. These primitive operations can be used directly as synchronization tools, or they can be used to form the foundation of more abstract synchronization mechanisms.

Memory barriers

In Section 5.3, we saw that a system may reorder instructions, a policy that can lead to unreliable data states. How a computer architecture determines what memory guarantees it will provide to an application program is known as its ***memory model***. In general, a memory model falls into one of two categories:

1. **Strongly ordered**, where a memory modification on one processor is immediately visible to all other processors.
2. **Weakly ordered**, where modifications to memory on one processor may not be immediately visible to other processors.

Memory models vary by processor type, so kernel developers cannot make any assumptions regarding the visibility of modifications to memory on a shared-memory multiprocessor. To address this issue, computer architectures provide instructions that can *force* any changes in memory to be propagated to all other

processors, thereby ensuring that memory modifications are visible to threads running on other processors. Such instructions are known as ***memory barriers*** or ***memory fences***. When a memory barrier instruction is performed, the system ensures that all loads and stores are completed before any subsequent load or store operations are performed. Therefore, even if instructions were reordered, the memory barrier ensures that the store operations are completed in memory and visible to other processors before future load or store operations are performed.

Let's return to our most recent example, in which reordering of instructions could have resulted in the wrong output, and use a memory barrier to ensure that we obtain the expected output.

If we add a memory barrier operation to Thread 1

```
while (!flag)
  memory_barrier();
print x;
```

we guarantee that the value of `flag` is loaded before the value of `x`.

Similarly, if we place a memory barrier between the assignments performed by Thread 2

```
x = 100;
memory_barrier();
flag = true;
```

we ensure that the assignment to `x` occurs before the assignment to `flag`.

With respect to Peterson's solution, we could place a memory barrier between the first two assignment statements in the entry section to avoid the reordering of operations shown in Figure 5.3.2. Note that memory barriers are considered very low-level operations and are typically only used by kernel developers when writing specialized code that ensures mutual exclusion.

Hardware instructions

Many modern computer systems provide special hardware instructions that allow us either to test and modify the content of a word or to swap the contents of two words ***atomically***—that is, as one uninterruptible unit. We can use these special instructions to solve the critical-section problem in a relatively simple manner. Rather than discussing one specific instruction for one specific machine, we abstract the main concepts behind these types of instructions by describing the `test_and_set()` and `compare_and_swap()` instructions.

The `test_and_set()` instruction can be defined as shown in Figure 5.4.1. The important characteristic of this instruction is that it is executed atomically. Thus, if two `test_and_set()` instructions are executed simultaneously (each on a different core), they will be executed sequentially in some arbitrary order. If the machine supports the `test_and_set()` instruction, then we can implement mutual exclusion by declaring a boolean variable `lock`, initialized to `false`. The structure of process P_i is shown in Figure 5.4.2.

Figure 5.4.1: The definition of the atomic test_and_set() instruction.

```
boolean test_and_set(boolean *target) {
  boolean rv = *target;
  *target = true;

  return rv;
}
```


Figure 5.4.2: Mutual-exclusion implementation with test_and_set().

```
do {
  while (test_and_set(&lock))
     ; /* do nothing */

     /* critical section */

  lock = false;

     /* remainder section */
} while (true);
```

The **compare_and_swap()** instruction (CAS), just like the **test_and_set()** instruction, operates on two words atomically, but uses a different mechanism that is based on swapping the content of two words.

The CAS instruction operates on three operands and is defined in Figure 5.4.3. The operand **value** is set to **new_value** only if the expression **(*value == expected)** is true. Regardless, CAS always returns the original value of the variable **value**. The important characteristic of this instruction is that it is executed atomically. Thus, if two CAS instructions are executed simultaneously (each on a different core), they will be executed sequentially in some arbitrary order.

Figure 5.4.3: The definition of the atomic compare_and_swap() instruction.

```
int compare_and_swap(int *value, int expected, int new_value) {
  int temp = *value;

  if (*value == expected)
    *value = new_value;

  return temp;
}
```


Mutual exclusion using CAS can be provided as follows: A global variable (**lock**) is declared and is initialized to 0. The first process that invokes **compare_and_swap()** will set **lock** to 1. It will then enter

its critical section, because the original value of `lock` was equal to the expected value of 0. Subsequent calls to `compare_and_swap()` will not succeed, because `lock` now is not equal to the expected value of 0. When a process exits its critical section, it sets `lock` back to 0, which allows another process to enter its critical section. The structure of process P_i is shown in Figure 5.4.4.

Figure 5.4.4: Mutual exclusion with the compare_and_swap() instruction.

```
while (true) {
  while (compare_and_swap(&lock, 0, 1) != 0)
     ; /* do nothing */

     /* critical section */

     lock = 0;

     /* remainder section */
}
```


PARTICIPATION ACTIVITY 5.4.1: Mid-section review question.

If lock = 0, the function call `compare_and_swap`(&lock,0,1) sets lock to ____ and returns _____.

- 0, 0
- 0, 1
- 1, 0
- 1, 1

Although this algorithm satisfies the mutual-exclusion requirement, it does not satisfy the bounded-waiting requirement. In Figure 5.4.5, we present another algorithm using the `compare_and_swap()` instruction that satisfies all the critical-section requirements. The common data structures are

```
boolean waiting[n];
int lock;
```


The elements in the `waiting` array are initialized to `false`, and `lock` is initialized to 0. To prove that the mutual-exclusion requirement is met, we note that process P_i can enter its critical section only if either `waiting[i]` == `false` or `key` == `0`. The value of `key` can become `0` only if the `compare_and_swap()` is executed. The first process to execute the `compare_and_swap()` will find `key` == `0`; all others must wait. The variable `waiting[i]` can become `false` only if another process leaves its critical section; only one `waiting[i]` is set to `false`, maintaining the mutual-exclusion requirement.

Figure 5.4.5: Bounded-waiting mutual exclusion with compare_and_swap().

```
while (true) {
  waiting[i] = true;
  key = 1;
  while (waiting[i] && key == 1)
    key = compare_and_swap(&lock,0,1);
  waiting[i] = false;

    /* critical section */

  j = (i + 1) % n;
  while ((j != i) && !waiting[j])
    j = (j + 1) % n;

  if (j == i)
    lock = 0;
  else
     waiting[j] = false;

     /* remainder section */
}
```


Making compare-and-swap atomic

On Intel x86 architectures, the assembly language statement **cmpxchg** is used to implement the **compare_and_swap()** instruction. To enforce atomic execution, the **lock** prefix is used to lock the bus while the destination operand is being updated. The general form of this instruction appears as:

```
lock cmpxchg <destination operand>, <source operand>
```

To prove that the progress requirement is met, we note that the arguments presented for mutual exclusion also apply here, since a process exiting the critical section either sets **lock** to **0** or sets **waiting[j]** to **false**. Both allow a process that is waiting to enter its critical section to proceed.

To prove that the bounded-waiting requirement is met, we note that, when a process leaves its critical section, it scans the array **waiting** in the cyclic ordering ($i + 1, i + 2, \ldots, n - 1, 0, \ldots, i - 1$). It designates the first process in this ordering that is in the entry section (**waiting[j]** == **true**) as the next one to enter the critical section. Any process waiting to enter its critical section will thus do so within $n - 1$ turns.

Details describing the implementation of the atomic **test_and_set()** and **compare_and_swap()** instructions are discussed more fully in books on computer architecture.

Atomic variables

Typically, the `compare_and_swap()` instruction is not used directly to provide mutual exclusion. Rather, it is used as a basic building block for constructing other tools that solve the critical-section problem. One such tool is an ***atomic variable***, which provides atomic operations on basic data types such as integers and booleans. We know from Section 5.1 that incrementing or decrementing an integer value may produce a race condition. Atomic variables can be used to ensure mutual exclusion in situations where there may be a data race on a single variable while it is being updated, as when a counter is incremented.

Most systems that support atomic variables provide special atomic data types as well as functions for accessing and manipulating atomic variables. These functions are often implemented using `compare_and_swap()` operations. As an example, the following increments the atomic integer `sequence`:

```
increment(&sequence);
```

where the `increment()` function is implemented using the CAS instruction:

```
void increment(atomic_int *v)
{
  int temp;

  do {
    temp = *v;
  }
  while (temp != compare_and_swap(v, temp, temp+1));
}
```

It is important to note that although atomic variables provide atomic updates, they do not entirely solve race conditions in all circumstances. For example, in the bounded-buffer problem described in Section 5.1, we could use an atomic integer for `count`. This would ensure that the updates to `count` were atomic. However, the producer and consumer processes also have `while` loops whose condition depends on the value of `count`. Consider a situation in which the buffer is currently empty and two consumers are looping while waiting for `count` > 0. If a producer entered one item in the buffer, both consumers could exit their `while` loops (as `count` would no longer be equal to 0) and proceed to consume, even though the value of `count` was only set to 1.

Atomic variables are commonly used in operating systems as well as concurrent applications, although their use is often limited to single updates of shared data such as counters and sequence generators. In the following sections, we explore more robust tools that address race conditions in more generalized situations.

PARTICIPATION ACTIVITY 5.4.2: Section review questions.

1) A _____ instruction ensures that all loads and store operations are completed before additional load and store operations are performed.

- memory barrier
- atomic variable
- memory model

2) An instruction that executes atomically ____.

- must consist of only one machine instruction
- executes as a single, uninterruptible unit
- cannot be used to solve the critical section problem

Section glossary

memory model: Computer architecture memory guarantee, usually either strongly ordered or weakly ordered.

memory barriers: Computer instructions that force any changes in memory to be propagated to all other processors in the system.

memory fences: Computer instructions that force any changes in memory to be propagated to all other processors in the system.

atomically: A computer activity (such as a CPU instruction) that operates as one uninterruptable unit.

atomic variable: A programming language construct that provides atomic operations on basic data types such as integers and booleans.

5.5 Mutex locks

The hardware-based solutions to the critical-section problem presented in Section 5.4 are complicated as well as generally inaccessible to application programmers. Instead, operating-system designers build

higher-level software tools to solve the critical-section problem. The simplest of these tools is the ***mutex lock***. (In fact, the term ***mutex*** is short for ***mutual exclusion***.) We use the mutex lock to protect critical sections and thus prevent race conditions. That is, a process must acquire the lock before entering a critical section; it releases the lock when it exits the critical section. The `acquire()` function acquires the lock, and the `release()` function releases the lock, as illustrated in Figure 5.5.1.

Figure 5.5.1: Solution to the critical-section problem using mutex locks.

```
while (true) {

   acquire lock

       critical section

   release lock

       remainder section

}
```

A mutex lock has a boolean variable `available` whose value indicates if the lock is available or not. If the lock is available, a call to `acquire()` succeeds, and the lock is then considered unavailable. A process that attempts to acquire an unavailable lock is blocked until the lock is released.

The definition of `acquire()` is as follows:

```
acquire() {
   while (!available)
     ; /* busy wait */
   available = false;
}
```

The definition of `release()` is as follows:

```
release() {
  available = true;
}
```

Calls to either `acquire()` or `release()` must be performed atomically. Thus, mutex locks can be implemented using the CAS operation described in Section 5.4, and we leave the description of this technique as an exercise.

PARTICIPATION ACTIVITY 5.5.1: The critical section problem.

Provide the correct ordering for solving the critical section problem using mutex locks.

How to use this tool

Unused blocks

- Critical Section
- release()
- acquire()
- Remainder Section

Proof Reset

Move blocks here

0 of 4 blocks correct

PARTICIPATION ACTIVITY 5.5.2: Section review questions.

There is the shared global variable

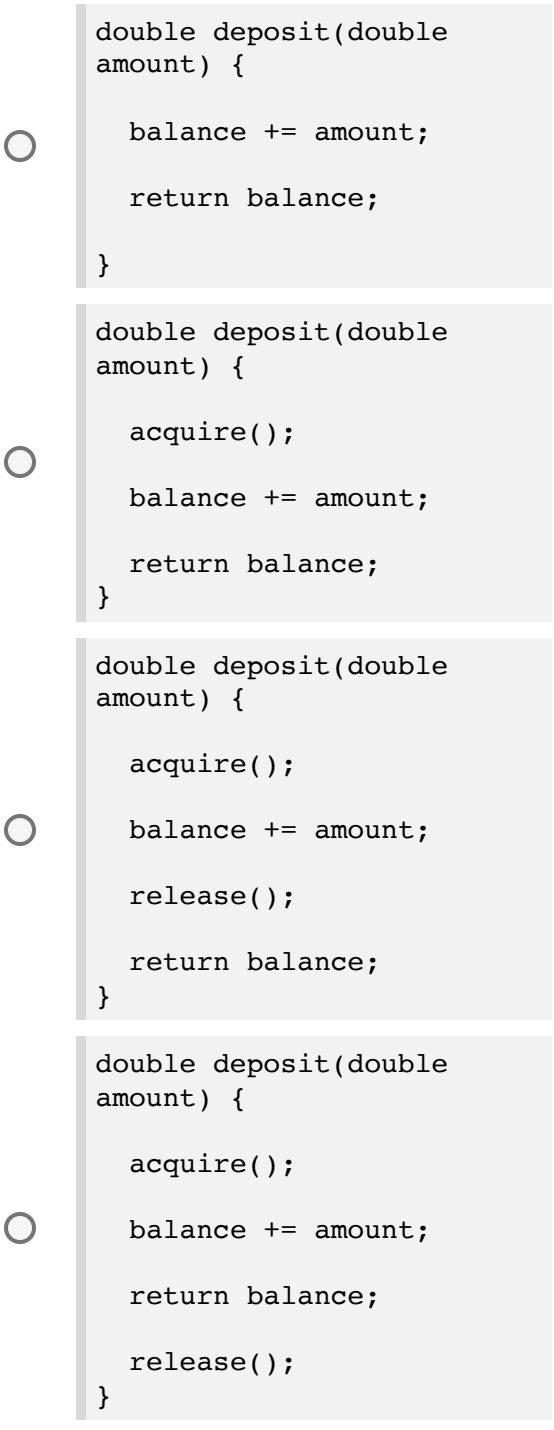
```
double balance;
```

that represents the balance in a banking account. There are multiple threads that may concurrently call the `deposit()` and `withdraw()` functions

```
double deposit(double amount) {
  balance += amount;
  return balance;
}

double withdraw(double amount) {
  if (amount <= balance) {
    balance -= amount;
  }
  return balance;
}
```


There is a race condition on the shared variable `balance`.

1) Which of the following code statements correctly fixes the race condition in the `deposit()` function using a mutex lock with the `acquire()` and `release()` operations?



```
double deposit(double
amount) {

  balance += amount;

  return balance;

}
```

```
double deposit(double
amount) {

  acquire();

  balance += amount;

  return balance;
}
```

```
double deposit(double
amount) {

  acquire();

  balance += amount;

  release();

  return balance;
}
```

```
double deposit(double
amount) {

  acquire();

  balance += amount;

  return balance;

  release();
}
```

2) Which of the following code statements correctly fixes the race condition in the `withdraw()` function using a mutex lock with the `acquire()` and `release()` operations?

- ○

```
double withdraw(double
amount) {

  if (amount <= balance) {
    acquire();
    balance -= amount;
    release();
  }

  return balance;
}
```

- ○

```
double withdraw(double
amount) {

  acquire();

  if (amount <= balance) {
    balance -= amount;

    release();
  }

  return balance;
}
```

- ○

```
double withdraw(double
amount) {

  if (amount <= balance) {
    acquire();
    balance -= amount;
  }

  release();

  return balance;
}
```

- ○

```
double withdraw(double
amount) {

  acquire();

  if (amount <= balance) {
    balance -= amount;
  }

  release();
```

```
    return balance;
}
```

Lock contention

Locks are either contended or uncontended. A lock is considered ***contended*** if a thread blocks while trying to acquire the lock. If a lock is available when a thread attempts to acquire it, the lock is considered ***uncontended***. Contended locks can experience either ***high contention*** (a relatively large number of threads attempting to acquire the lock) or ***low contention*** (a relatively small number of threads attempting to acquire the lock.) Unsurprisingly, highly contended locks tend to decrease overall performance of concurrent applications.

What is meant by 'short duration'?

Spinlocks are often identified as the locking mechanism of choice on multiprocessor systems when the lock is to be held for a short duration. But what exactly constitutes a *short duration*? Given that waiting on a lock requires two context switches—a context switch to move the thread to the waiting state and a second context switch to restore the waiting thread once the lock becomes available—the general rule is to use a spinlock if the lock will be held for a duration of less than two context switches.

The main disadvantage of the implementation given here is that it requires ***busy waiting***. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to `acquire()`. This continual looping is clearly a problem in a real multiprogramming system, where a single CPU core is shared among many processes. Busy waiting also wastes CPU cycles that some other process might be able to use productively. (In Section 5.6, we examine a strategy that avoids busy waiting by temporarily putting the waiting process to sleep and then awakening it once the lock becomes available.)

The type of mutex lock we have been describing is also called a ***spinlock*** because the process "spins" while waiting for the lock to become available. (We see the same issue with the code examples illustrating the `compare_and_swap()` instruction.) Spinlocks do have an advantage, however, in that no context switch is required when a process must wait on a lock, and a context switch may take considerable time. In certain circumstances on multicore systems, spinlocks are in fact the preferable choice for locking. If a lock is to be held for a short duration, one thread can "spin" on one processing core while another thread performs its critical section on another core. On modern multicore computing systems, spinlocks are widely used in many operating systems.

In the chapter Synchronization Examples, we examine how mutex locks can be used to solve classical synchronization problems. We also discuss how mutex locks and spinlocks are used in several operating systems, as well as in Pthreads.

PARTICIPATION ACTIVITY 5.5.3: Section review questions.

1) A mutex lock ____.

- is another term for a boolean variable
- can be used to solve the critical section problem
- is not guaranteed to have atomic operations

2) What is the correct order of operations for protecting a critical section using mutex locks?

- `release()` followed by `acquire()`
- `acquire()` followed by `release()`
- Setting the lock to true followed by setting the lock to false

3) What is the best scenario for implementing a mutex lock as a spinlock?

- On a single-core system where the lock is held for a short duration.
- On a multi-core system where the lock is held for a long duration.
- On a multi-core system where the lock is held for a short duration.

Section glossary

mutex lock: A mutual exclusion lock; the simplest software tool for assuring mutual exclusion.

contended: A term describing the condition of a lock when a thread blocks while trying to acquire it.

uncontended: A term describing a lock that is available when a thread attempts to acquire it.

busy waiting: A practice that allows a thread or process to use CPU time continuously while waiting for something. An I/O loop in which an I/O thread continuously reads status information while waiting for I/O to complete.

spinlock: A locking mechanism that continuously uses the CPU while waiting for access to the lock.

5.6 Semaphores

Mutex locks, as we mentioned earlier, are generally considered the simplest of synchronization tools. In this section, we examine a more robust tool that can behave similarly to a mutex lock but can also provide more sophisticated ways for processes to synchronize their activities.

A ***semaphore*** `S` is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: `wait()` and `signal()`. Semaphores were introduced by the Dutch computer scientist Edsger Dijkstra, and such, the `wait()` operation was originally termed `P` (from the Dutch ***proberen***, "to test"); `signal()` was originally called `V` (from ***verhogen***, "to increment"). The definition of `wait()` is as follows:

```
wait(S) {
   while (S <= 0)
     ; /* busy wait */
     S--;
}
```

The definition of `signal()` is as follows:

```
signal(S) {
   S++;
}
```

All modifications to the integer value of the semaphore in the `wait()` and `signal()` operations must be executed atomically. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value. In addition, in the case of `wait(S)`, the testing of the integer value of `S` ($S \leq 0$), as well as its possible modification (`S--`), must be executed without interruption. We shall see how these operations can be implemented in Section Semaphore implementation. First, let's see how semaphores can be used.

Semaphore usage

Operating systems often distinguish between counting and binary semaphores. The value of a ***counting semaphore*** can range over an unrestricted domain. The value of a ***binary semaphore*** can range only between 0 and 1. Thus, binary semaphores behave similarly to mutex locks. In fact, on systems that do not provide mutex locks, binary semaphores can be used instead for providing mutual exclusion.

PARTICIPATION ACTIVITY 5.6.1: Mid-section review question.

To use a binary semaphore to solve the critical section problem, the semaphore must be initialized to 1 and _____ must be called before entering a critical section and _____ must be called after exiting the critical section.

- `wait()`, `signal()`
- `signal()`, `wait()`
- Mutex locks - not semaphores - are used to solve the critical section problem.

Counting semaphores can be used to control access to a given resource consisting of a finite number of instances. The semaphore is initialized to the number of resources available. Each process that wishes to use a resource performs a `wait()` operation on the semaphore (thereby decrementing the count). When a process releases a resource, it performs a `signal()` operation (incrementing the count). When the count for the semaphore goes to 0, all resources are being used. After that, processes that wish to use a resource will block until the count becomes greater than 0.

We can also use semaphores to solve various synchronization problems. For example, consider two concurrently running processes: P_1 with a statement S_1 and P_2 with a statement S_2. Suppose we require that S_2 be executed only after S_1 has completed. We can implement this scheme readily by letting P_1 and P_2 share a common semaphore `synch`, initialized to 0. In process P_1, we insert the statements

```
S1;
signal(synch);
```

In process P_2, we insert the statements

```
wait(synch);
S2;
```

Because `synch` is initialized to 0, P_2 will execute S_2 only after P_1 has invoked `signal(synch)`, which is after statement S_1 has been executed.

PARTICIPATION ACTIVITY | 5.6.2: Mid-section review question.

A counting semaphore can be used as a binary semaphore.

- True
- False

Semaphore implementation

Recall that the implementation of mutex locks discussed in Section 5.5 suffers from busy waiting. The definitions of the `wait()` and `signal()` semaphore operations just described present the same problem. To overcome this problem, we can modify the definition of the `wait()` and `signal()` operations as follows: When a process executes the `wait()` operation and finds that the semaphore value is not positive, it must wait. However, rather than engaging in busy waiting, the process can suspend itself. The suspend operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state. Then control is transferred to the CPU scheduler, which selects another process to execute.

A process that is suspended, waiting on a semaphore `S`, should be restarted when some other process executes a `signal()` operation. The process is restarted by a `wakeup()` operation, which changes the process from the waiting state to the ready state. The process is then placed in the ready queue. (The CPU may or may not be switched from the running process to the newly ready process, depending on the CPU-scheduling algorithm.)

To implement semaphores under this definition, we define a semaphore as follows:

```
typedef struct {
    int value;
    struct process *list;
} semaphore;
```

Each semaphore has an integer `value` and a list of processes `list`. When a process must wait on a semaphore, it is added to the list of processes. A `signal()` operation removes one process from the list of waiting processes and awakens that process.

Now, the `wait()` semaphore operation can be defined as

```
wait(semaphore *S) {
     S->value--;
     if (S->value < 0) {
            add this process to S->list;
            sleep();
     }
}
```

and the **signal()** semaphore operation can be defined as

```
signal(semaphore *S) {
     S->value++;
     if (S->value <= 0) {
            remove a process P from S->list;
            wakeup(P);
     }
}
```


The **sleep()** operation suspends the process that invokes it. The **wakeup(P)** operation resumes the execution of a suspended process **P**. These two operations are provided by the operating system as basic system calls.

PARTICIPATION ACTIVITY 5.6.3: Mid-section review question.

_____ can be used to prevent busy waiting when implementing a semaphore.

Check Show answer

Note that in this implementation, semaphore values may be negative, whereas semaphore values are never negative under the classical definition of semaphores with busy waiting. If a semaphore value is negative, its magnitude is the number of processes waiting on that semaphore. This fact results from switching the order of the decrement and the test in the implementation of the **wait()** operation.

The list of waiting processes can be easily implemented by a link field in each process control block (PCB). Each semaphore contains an integer value and a pointer to a list of PCBs. One way to add and remove processes from the list so as to ensure bounded waiting is to use a FIFO queue, where the semaphore contains both head and tail pointers to the queue. In general, however, the list can use any queuing strategy. Correct usage of semaphores does not depend on a particular queuing strategy for the semaphore lists.

As mentioned, it is critical that semaphore operations be executed atomically. We must guarantee that no two processes can execute **wait()** and **signal()** operations on the same semaphore at the same time. This is a critical-section problem, and in a single-processor environment, we can solve it by simply inhibiting interrupts during the time the **wait()** and **signal()** operations are executing. This scheme works in a single-processor environment because, once interrupts are inhibited, instructions from different processes cannot be interleaved. Only the currently running process executes until interrupts are reenabled and the scheduler can regain control.

In a multicore environment, interrupts must be disabled on every processing core. Otherwise, instructions from different processes (running on different cores) may be interleaved in some arbitrary way. Disabling

interrupts on every core can be a difficult task and can seriously diminish performance. Therefore, SMP systems must provide alternative techniques—such as `compare_and_swap()` or spinlocks—to ensure that `wait()` and `signal()` are performed atomically.

It is important to admit that we have not completely eliminated busy waiting with this definition of the `wait()` and `signal()` operations. Rather, we have moved busy waiting from the entry section to the critical sections of application programs. Furthermore, we have limited busy waiting to the critical sections of the `wait()` and `signal()` operations, and these sections are short (if properly coded, they should be no more than about ten instructions). Thus, the critical section is almost never occupied, and busy waiting occurs rarely, and then for only a short time. An entirely different situation exists with application programs whose critical sections may be long (minutes or even hours) or may almost always be occupied. In such cases, busy waiting is extremely inefficient.

PARTICIPATION ACTIVITY 5.6.4: Section review questions.

1) A semaphore _____ .

- contains an integer value
- is not guaranteed to have atomic operations
- Uses the same `acquire()` and `release()` operations as a mutex lock.

2) Which of the following statements are false?

- Semaphores can be used to solve the critical section problem.
- Semaphores can be used to control access to a finite number of resources.
- Semaphores are equivalent to mutex locks.

Section glossary

semaphore: An integer variable that, apart from initialization, is accessed only through two standard atomic operations: wait() and signal().

counting semaphore: A semaphore that has a value between 0 and N, to control access to a resource with N instances.

binary semaphore: A semaphore of values 0 and 1 that limits access to one resource (acting similarly to a mutex lock).

5.7 Monitors

Although semaphores provide a convenient and effective mechanism for process synchronization, using them incorrectly can result in timing errors that are difficult to detect, since these errors happen only if particular execution sequences take place, and these sequences do not always occur.

We have seen an example of such errors in the use of a count in our solution to the producer-consumer problem (Section 5.1). In that example, the timing problem happened only rarely, and even then the **`count`** value appeared to be reasonable—off by only 1. Nevertheless, the solution is obviously not an acceptable one. It is for this reason that mutex locks and semaphores were introduced in the first place.

Unfortunately, such timing errors can still occur when either mutex locks or semaphores are used. To illustrate how, we review the semaphore solution to the critical-section problem. All processes share a binary semaphore variable **`mutex`**, which is initialized to 1. Each process must execute **`wait(mutex)`** before entering the critical section and **`signal(mutex)`** afterward. If this sequence is not observed, two processes may be in their critical sections simultaneously. Next, we list several difficulties that may result. Note that these difficulties will arise even if a ***single*** process is not well behaved. This situation may be caused by an honest programming error or an uncooperative programmer.

- Suppose that a program interchanges the order in which the **`wait()`** and **`signal()`** operations on the semaphore **`mutex`** are executed, resulting in the following execution:

```
signal(mutex);
   …
  critical section
   …
wait(mutex);
```

  In this situation, several processes may be executing in their critical sections simultaneously, violating the mutual-exclusion requirement. This error may be discovered only if several processes are simultaneously active in their critical sections. Note that this situation may not always be reproducible.

- Suppose that a program replaces **`signal(mutex)`** with **`wait(mutex)`**. That is, it executes

```
wait(mutex);
    …
  critical section
```

```
    …
wait(mutex);
```

In this case, the process will permanently block on the second call to **`wait()`**, as the semaphore is now unavailable.

- Suppose that a process omits the **`wait(mutex)`**, or the **`signal(mutex)`**, or both. In this case, either mutual exclusion is violated or the process will permanently block.

These examples illustrate that various types of errors can be generated easily when programmers use semaphores or mutex locks incorrectly to solve the critical-section problem. One strategy for dealing with such errors is to incorporate simple synchronization tools as high-level language constructs. In this section, we describe one fundamental high-level synchronization construct—the ***monitor*** type.

Monitor usage

An ***abstract data type***—or *ADT*—encapsulates data with a set of functions to operate on that data that are independent of any specific implementation of the ADT. A ***monitor type*** is an ADT that includes a set of programmer-defined operations that are provided with mutual exclusion within the monitor. The monitor type also declares the variables whose values define the state of an instance of that type, along with the bodies of functions that operate on those variables. The syntax of a monitor type is shown in Figure 5.7.1. The representation of a monitor type cannot be used directly by the various processes. Thus, a function defined within a monitor can access only those variables declared locally within the monitor and its formal parameters. Similarly, the local variables of a monitor can be accessed by only the local functions.

Figure 5.7.1: Pseudocode syntax of a monitor.

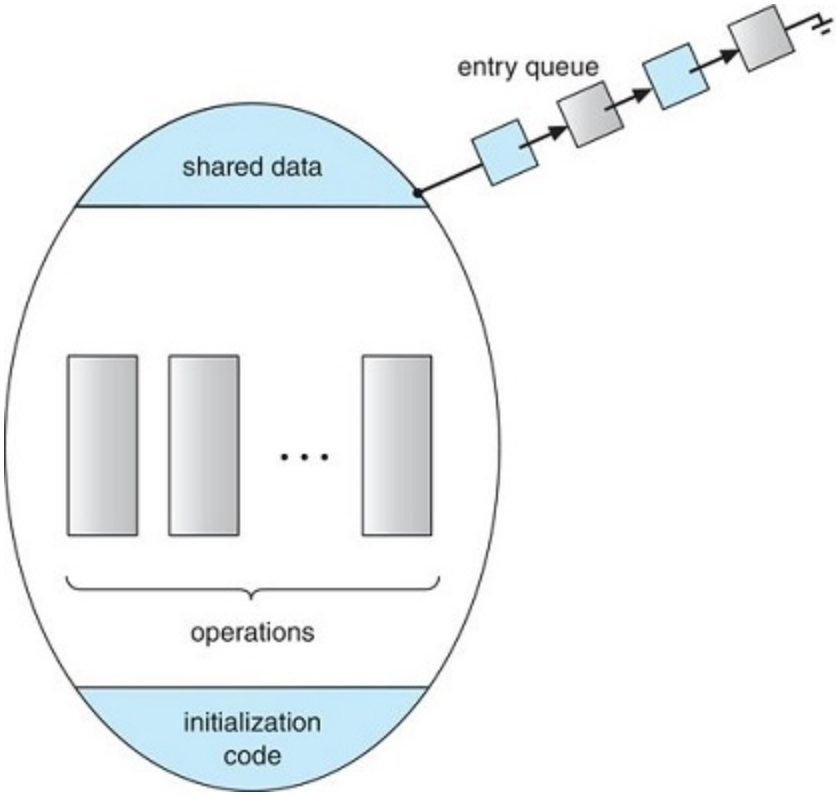
```
monitor monitor name
{
  /* shared variable declarations */

  function P1 ( . . . ) {
    . . .
  }

  function P2 ( . . . ) {
    . . .
  }

        .
        .
        .
  function Pn ( . . . ) {
    . . .
  }

  initialization_code ( . . . ) {
    . . .
```

```
    }
}
```

The monitor construct ensures that only one process at a time is active within the monitor. Consequently, the programmer does not need to code this synchronization constraint explicitly (Figure 5.7.2). However, the monitor construct, as defined so far, is not sufficiently powerful for modeling some synchronization schemes. For this purpose, we need to define additional synchronization mechanisms. These mechanisms are provided by the **condition** construct. A programmer who needs to write a tailor-made synchronization scheme can define one or more variables of type *condition*:

```
condition x, y;
```

Figure 5.7.2: Schematic view of a monitor.



The only operations that can be invoked on a condition variable are **wait()** and **signal()**. The operation

```
x.wait();
```

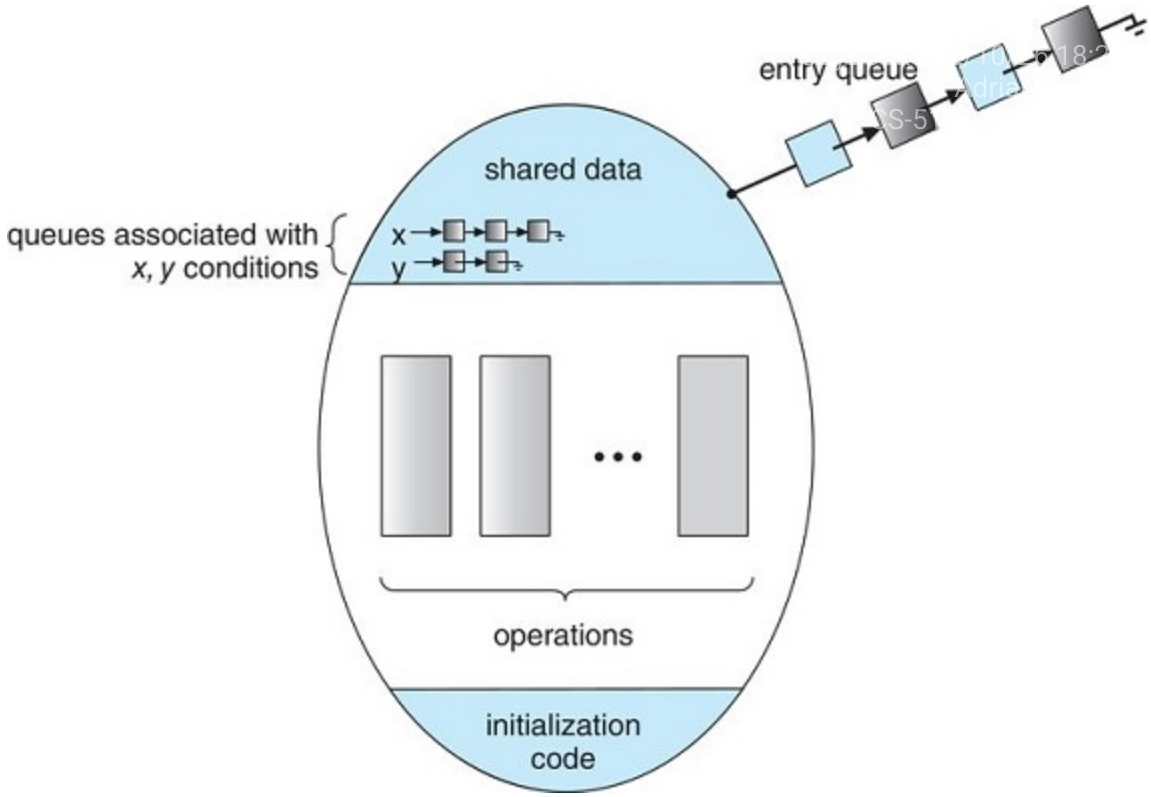

means that the process invoking this operation is suspended until another process invokes

```
x.signal();
```

The **x.signal()** operation resumes exactly one suspended process. If no process is suspended, then the **signal()** operation has no effect; that is, the state of **x** is the same as if the operation had never

been executed (Figure 5.7.3). Contrast this operation with the `signal()` operation associated with semaphores, which always affects the state of the semaphore.

Figure 5.7.3: Monitor with condition variables.



Now suppose that, when the `x.signal()` operation is invoked by a process P, there exists a suspended process Q associated with condition `x`. Clearly, if the suspended process Q is allowed to resume its execution, the signaling process P must wait. Otherwise, both P and Q would be active simultaneously within the monitor. Note, however, that conceptually both processes can continue with their execution. Two possibilities exist:

1. **Signal and wait.** P either waits until Q leaves the monitor or waits for another condition.
2. **Signal and continue.** Q either waits until P leaves the monitor or waits for another condition.

There are reasonable arguments in favor of adopting either option. On the one hand, since P was already executing in the monitor, the ***signal-and-continue*** method seems more reasonable. On the other, if we allow thread P to continue, then by the time Q is resumed, the logical condition for which Q was waiting may no longer hold. A compromise between these two choices exists as well: when thread P executes the `signal` operation, it immediately leaves the monitor. Hence, Q is immediately resumed.

Many programming languages have incorporated the idea of the monitor as described in this section, including Java and C#. Other languages—such as Erlang—provide concurrency support using a similar mechanism.

Implementing a monitor using semaphores

We now consider a possible implementation of the monitor mechanism using semaphores. For each monitor, a binary semaphore **`mutex`** (initialized to 1) is provided to ensure mutual exclusion. A process must execute **`wait(mutex)`** before entering the monitor and must execute **`signal(mutex)`** after leaving the monitor.

We will use the signal-and-wait scheme in our implementation. Since a signaling process must wait until the resumed process either leaves or waits, an additional binary semaphore, **`next`**, is introduced, initialized to 0. The signaling processes can use **`next`** to suspend themselves. An integer variable **`next_count`** is also provided to count the number of processes suspended on **`next`**. Thus, each external function **`F`** is replaced by

```
wait(mutex);
    …
  body of F
    …
if (next_count > 0)
  signal(next);
else
  signal(mutex);
```

Mutual exclusion within a monitor is ensured.

We can now describe how condition variables are implemented as well. For each condition **`x`**, we introduce a binary semaphore **`x_sem`** and an integer variable **`x_count`**, both initialized to 0. The operation **`x.wait()`** can now be implemented as

```
x_count++;
if (next_count > 0)
  signal(next);
else
  signal(mutex);
wait(x_sem);
x_count--;
```

The operation **`x.signal()`** can be implemented as

```
if (x_count > 0) {
  next_count++;
  signal(x_sem);
  wait(next);
  next_count--;
}
```


This implementation is applicable to the definitions of monitors given by both Hoare and Brinch-Hansen (see the bibliographical notes at the end of the chapter). In some cases, however, the generality of the implementation is unnecessary, and a significant improvement in efficiency is possible.

Resuming processes within a monitor

We turn now to the subject of process-resumption order within a monitor. If several processes are suspended on condition `x`, and an `x.signal()` operation is executed by some process, then how do we determine which of the suspended processes should be resumed next? One simple solution is to use a first-come, first-served (FCFS) ordering, so that the process that has been waiting the longest is resumed first. In many circumstances, however, such a simple scheduling scheme is not adequate. In these circumstances, the ***conditional-wait*** construct can be used. This construct has the form

```
x.wait(c);
```

where `c` is an integer expression that is evaluated when the `wait()` operation is executed. The value of `c`, which is called a ***priority number***, is then stored with the name of the process that is suspended. When `x.signal()` is executed, the process with the smallest priority number is resumed next.

To illustrate this new mechanism, consider the `ResourceAllocator` monitor shown in Figure 5.7.4, which controls the allocation of a single resource among competing processes. Each process, when requesting an allocation of this resource, specifies the maximum time it plans to use the resource. The monitor allocates the resource to the process that has the shortest time-allocation request. A process that needs to access the resource in question must observe the following sequence:

```
R.acquire(t);
     …
   access the resource;
     …
R.release();
```

where `R` is an instance of type `ResourceAllocator`.

Figure 5.7.4: A monitor to allocate a single resource.

```
monitor ResourceAllocator
{
  boolean busy;
  condition x;

  void acquire(int time) {
    if (busy)
      x.wait(time);
    busy = true;
  }

  void release() {
    busy = false;
    x.signal();
  }
  initialization_code() {
    busy = false;
```

```
    }
}
```

Unfortunately, the monitor concept cannot guarantee that the preceding access sequence will be observed. In particular, the following problems can occur:

- A process might access a resource without first gaining access permission to the resource.
- A process might never release a resource once it has been granted access to the resource.
- A process might attempt to release a resource that it never requested.
- A process might request the same resource twice (without first releasing the resource).

The same difficulties are encountered with the use of semaphores, and these difficulties are similar in nature to those that encouraged us to develop the monitor constructs in the first place. Previously, we had to worry about the correct use of semaphores. Now, we have to worry about the correct use of higher-level programmer-defined operations, with which the compiler can no longer assist us.

One possible solution to the current problem is to include the resource-access operations within the `ResourceAllocator` monitor. However, using this solution will mean that scheduling is done according to the built-in monitor-scheduling algorithm rather than the one we have coded.

To ensure that the processes observe the appropriate sequences, we must inspect all the programs that make use of the `ResourceAllocator` monitor and its managed resource. We must check two conditions to establish the correctness of this system. First, user processes must always make their calls on the monitor in a correct sequence. Second, we must be sure that an uncooperative process does not simply ignore the mutual-exclusion gateway provided by the monitor and try to access the shared resource directly, without using the access protocols. Only if these two conditions can be ensured can we guarantee that no time-dependent errors will occur and that the scheduling algorithm will not be defeated.

Although this inspection may be possible for a small, static system, it is not reasonable for a large system or a dynamic system. This access-control problem can be solved only through the use of the additional mechanisms that are described in the chapter Protection.

PARTICIPATION ACTIVITY 5.7.1: Section review questions.

1) A monitor _____ .

- represents programmer-defined data and a set of functions that provide mutual exclusion
- is similar to a semaphore
- allows multiple threads to be active within the monitor at the same time

2) A process that calls the `wait()` function on a condition variable is blocked _____.

- only if another process is active in the monitor
- until another process calls signal() on the condition variable
- only if at least 1 other process has also called `wait()` on the condition variable

Section glossary

monitor: A high-level language synchronization construct that protects variables from race conditions.

abstract data type (ADT): A programming construct that encapsulates data with a set of functions to operate on that data that are independent of any specific implementation of the ADT.

conditional-wait: A component of the monitor construct that allows for waiting on a variable with a priority number to indicate which process should get the lock next.

priority number: A number indicating the position of a process in a conditional-wait queue in a monitor construct.

5.8 System model

Introduction

In a multiprogramming environment, several threads may compete for a finite number of resources. A thread requests resources; if the resources are not available at that time, the thread enters a waiting state. Sometimes, a waiting thread can never again change state, because the resources it has requested are held by other waiting threads. This situation is called a ***deadlock***. We discussed this issue briefly in the chapter Synchronization Tools as a form of liveness failure. There, we defined deadlock as a situation in which *every process in a set of processes is waiting for an event that can be caused only by another process in the set*.

Perhaps the best illustration of a deadlock can be drawn from a law passed by the Kansas legislature early in the 20th century. It said, in part: "When two trains approach each other at a crossing, both shall come to

a full stop and neither shall start up again until the other has gone."

In this chapter, we describe methods that application developers as well as operating-system programmers can use to prevent or deal with deadlocks. Although some applications can identify programs that may deadlock, operating systems typically do not provide deadlock-prevention facilities, and it remains the responsibility of programmers to ensure that they design deadlock-free programs. Deadlock problems—as well as other liveness failures—are becoming more challenging as demand continues for increased concurrency and parallelism on multicore systems.

Chapter objectives

- Illustrate how deadlock can occur when mutex locks are used.
- Define the four necessary conditions that characterize deadlock.
- Identify a deadlock situation in a resource allocation graph.
- Evaluate the four different approaches for preventing deadlocks.
- Apply the banker's algorithm for deadlock avoidance.
- Apply the deadlock detection algorithm.
- Evaluate approaches for recovering from deadlock.

Section glossary

deadlock: The state in which two processes or threads are stuck waiting for an event that can only be caused by one of the processes or threads.

System model

A system consists of a finite number of resources to be distributed among a number of competing threads. The resources may be partitioned into several types (or classes), each consisting of some number of identical instances. CPU cycles, files, and I/O devices (such as network interfaces and DVD drives) are examples of resource types. If a system has four CPUs, then the resource type *CPU* has four instances. Similarly, the resource type *network* may have two instances. If a thread requests an instance of a resource type, the allocation of ***any*** instance of the type should satisfy the request. If it does not, then the instances are not identical, and the resource type classes have not been defined properly.

The various synchronization tools discussed in the chapter Synchronization Tools, such as mutex locks and semaphores, are also system resources; and on contemporary computer systems, they are the most

common sources of deadlock. However, definition is not a problem here. A lock is typically associated with a specific data structure—that is, one lock may be used to protect access to a queue, another to protect access to a linked list, and so forth. For that reason, each instance of a lock is typically assigned its own resource class.

Note that throughout this chapter we discuss kernel resources, but threads may use resources from other processes (for example, via interprocess communication), and those resource uses can also result in deadlock. Such deadlocks are not the concern of the kernel and thus not described here.

A thread must request a resource before using it and must release the resource after using it. A thread may request as many resources as it requires to carry out its designated task. Obviously, the number of resources requested may not exceed the total number of resources available in the system. In other words, a thread cannot request two network interfaces if the system has only one.

Under the normal mode of operation, a thread may utilize a resource in only the following sequence:

1. **Request.** The thread requests the resource. If the request cannot be granted immediately (for example, if a mutex lock is currently held by another thread), then the requesting thread must wait until it can acquire the resource.
2. **Use.** The thread can operate on the resource (for example, if the resource is a mutex lock, the thread can access its critical section).
3. **Release.** The thread releases the resource.

The request and release of resources may be system calls, as explained in the chapter Operating-System Structures. Examples are the `request()` and `release()` of a device, `open()` and `close()` of a file, and `allocate()` and `free()` memory system calls. Similarly, as we saw in the chapter Synchronization Tools, request and release can be accomplished through the `wait()` and `signal()` operations on semaphores and through `acquire()` and `release()` of a mutex lock. For each use of a kernel-managed resource by a thread, the operating system checks to make sure that the thread has requested and has been allocated the resource. A system table records whether each resource is free or allocated. For each resource that is allocated, the table also records the thread to which it is allocated. If a thread requests a resource that is currently allocated to another thread, it can be added to a queue of threads waiting for this resource.

A set of threads is in a deadlocked state when every thread in the set is waiting for an event that can be caused only by another thread in the set. The events with which we are mainly concerned here are resource acquisition and release. The resources are typically logical (for example, mutex locks, semaphores, and files); however, other types of events may result in deadlocks, including reading from a network interface or the IPC (interprocess communication) facilities discussed in the chapter Processes.

To illustrate a deadlocked state, we refer back to the dining-philosophers problem from Section The dining-philosophers problem. In this situation, resources are represented by chopsticks. If all the philosophers get hungry at the same time, and each philosopher grabs the chopstick on her left, there are no longer any available chopsticks. Each philosopher is then blocked waiting for her right chopstick to become available.

Developers of multithreaded applications must remain aware of the possibility of deadlocks. The locking tools presented in the chapter Synchronization Tools are designed to avoid race conditions. However, in using these tools, developers must pay careful attention to how locks are acquired and released. Otherwise, deadlock can occur, as described next.

PARTICIPATION ACTIVITY 5.8.1: Section review questions.

1) A deadlock occurs when ____.

- a process is waiting for I/O to a device that does not exist
- the system has no available free resources
- every process in a set is waiting for an event that can only be caused by another process in the set

2) Processes can wait while _____ a resource.

- requesting
- using
- releasing

5.9 Deadlock in multithreaded applications

Prior to examining how deadlock issues can be identified and managed, we first illustrate how deadlock can occur in a multithreaded Pthread program using POSIX mutex locks. The `pthread_mutex_init()` function initializes an unlocked mutex. Mutex locks are acquired and released using `pthread_mutex_lock()` and `pthread_mutex_unlock()`, respectively. If a thread attempts to acquire a locked mutex, the call to `pthread_mutex_lock()` blocks the thread until the owner of the mutex lock invokes `pthread_mutex_unlock()`.

Two mutex locks are created and initialized in the following code example:

```
pthread_mutex_t first_mutex;
pthread_mutex_t second_mutex;

pthread_mutex_init(&first_mutex,NULL);
pthread_mutex_init(&second_mutex,NULL);
```

Next, two threads—**thread_one** and **thread_two**—are created, and both these threads have access to both mutex locks. **thread_one** and **thread_two** run in the functions **do_work_one()** and **do_work_two()**, respectively, as shown in Figure 5.9.1.

Figure 5.9.1: Deadlock example.


```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
   pthread_mutex_lock(&first_mutex);
   pthread_mutex_lock(&second_mutex);
   /**
     * Do some work
     */
   pthread_mutex_unlock(&second_mutex);
   pthread_mutex_unlock(&first_mutex);

   pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
   pthread_mutex_lock(&second_mutex);
   pthread_mutex_lock(&first_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&first_mutex);
   pthread_mutex_unlock(&second_mutex);

   pthread_exit(0);
}
```

In this example, **thread_one** attempts to acquire the mutex locks in the order (1) **first_mutex**, (2) **second_mutex**. At the same time, **thread_two** attempts to acquire the mutex locks in the order (1) **second_mutex**, (2) **first_mutex**. Deadlock is possible if **thread_one** acquires **first_mutex** while **thread_two** acquires **second_mutex**.

Note that, even though deadlock is possible, it will not occur if **thread_one** can acquire and release the mutex locks for **first_mutex** and **second_mutex** before **thread_two** attempts to acquire the locks. And, of course, the order in which the threads run depends on how they are scheduled by the CPU scheduler. This example illustrates a problem with handling deadlocks: it is difficult to identify and test for deadlocks that may occur only under certain scheduling circumstances.

Livelock

Livelock is another form of liveness failure. It is similar to deadlock; both prevent two or more threads from proceeding, but the threads are unable to proceed for different reasons. Whereas deadlock occurs when

every thread in a set is blocked waiting for an event that can be caused only by another thread in the set, livelock occurs when a thread continuously attempts an action that fails. Livelock is similar to what sometimes happens when two people attempt to pass in a hallway: One moves to his right, the other to her left, still obstructing each other's progress. Then he moves to his left, and she moves to her right, and so forth. They aren't blocked, but they aren't making any progress.

Livelock can be illustrated with the Pthreads **`pthread_mutex_trylock()`** function, which attempts to acquire a mutex lock without blocking. The code example in Figure 5.9.2 rewrites the example from Figure 5.9.1 so that it now uses **`pthread_mutex_trylock()`**. This situation can lead to livelock if **`thread_one`** acquires **`first_mutex`**, followed by **`thread_two`** acquiring **`second_mutex`**. Each thread then invokes **`pthread_mutex_trylock()`**, which fails, releases their respective locks, and repeats the same actions indefinitely.

Figure 5.9.2: Livelock example.

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
   int done = 0;

   while (!done) {
      pthread_mutex_lock(&first_mutex);
      if (pthread_mutex_trylock(&second_mutex)) {
         /**
          * Do some work
          */
         pthread_mutex_unlock(&second_mutex);
         pthread_mutex_unlock(&first_mutex);
         done = 1;
     }
     else
         pthread_mutex_unlock(&first_mutex);
   }
   pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
   int done = 0;

   while (!done) {
     pthread_mutex_lock(&second_mutex);
     if (pthread_mutex_trylock(&first_mutex)) {
         /**
          * Do some work
          */
         pthread_mutex_unlock(&first_mutex);
         pthread_mutex_unlock(&second_mutex);
         done = 1;
     }
     else
         pthread_mutex_unlock(&second_mutex);
```

```
    }

    pthread_exit(0);
}
```

Livelock typically occurs when threads retry failing operations at the same time. It thus can generally be avoided by having each thread retry the failing operation at random times. This is precisely the approach taken by Ethernet networks when a network collision occurs. Rather than trying to retransmit a packet immediately after a collision occurs, a host involved in a collision will ***backoff*** a random period of time before attempting to transmit again.

Livelock is less common than deadlock but nonetheless is a challenging issue in designing concurrent applications, and like deadlock, it may only occur under specific scheduling circumstances.

PARTICIPATION ACTIVITY 5.9.1: Section review questions.

The code example below _____ lead to deadlock between two threads.

```
/* thread_one runs in this function
*/
void *do_work_one(void *param)
{
   pthread_mutex_lock(&first_mutex);

pthread_mutex_lock(&second_mutex);
   /**
     * Do some work
     */

pthread_mutex_unlock(&second_mutex);

pthread_mutex_unlock(&first_mutex);

   pthread_exit(0);
}

/* thread_two runs in this function
*/
void *do_work_two(void *param)
{

pthread_mutex_lock(&second_mutex);
   pthread_mutex_lock(&first_mutex);
   /**
    * Do some work
    */

pthread_mutex_unlock(&first_mutex);

pthread_mutex_unlock(&second_mutex);

   pthread_exit(0);
}
```

- ○ will not
- ○ can
- ○ will

Section glossary

livelock: A condition in which a thread continuously attempts an action that fails.

5.10 Deadlock characterization

In the previous section we illustrated how deadlock could occur in multithreaded programming using mutex locks. We now look more closely at conditions that characterize deadlock.

Necessary conditions

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

1. **Mutual exclusion.** At least one resource must be held in a nonsharable mode; that is, only one thread at a time can use the resource. If another thread requests that resource, the requesting thread must be delayed until the resource has been released.
2. **Hold and wait.** A thread must be holding at least one resource and waiting to acquire additional resources that are currently being held by other threads.
3. **No preemption.** Resources cannot be preempted; that is, a resource can be released only voluntarily by the thread holding it, after that thread has completed its task.
4. **Circular wait.** A set $\{T_0, T_1, \ldots, T_n\}$ of waiting threads must exist such that T_0 is waiting for a resource held by T_1, T_1 is waiting for a resource held by T_2, ..., T_{n-1} is waiting for a resource held by T_n, and T_n is waiting for a resource held by T_0.

We emphasize that all four conditions must hold for a deadlock to occur. The circular-wait condition implies the hold-and-wait condition, so the four conditions are not completely independent. We shall see in Section 5.12, however, that it is useful to consider each condition separately.

PARTICIPATION ACTIVITY 5.10.1: Mid-section review question.

The four necessary conditions of deadlock must be present for deadlock to occur.

- True
- False

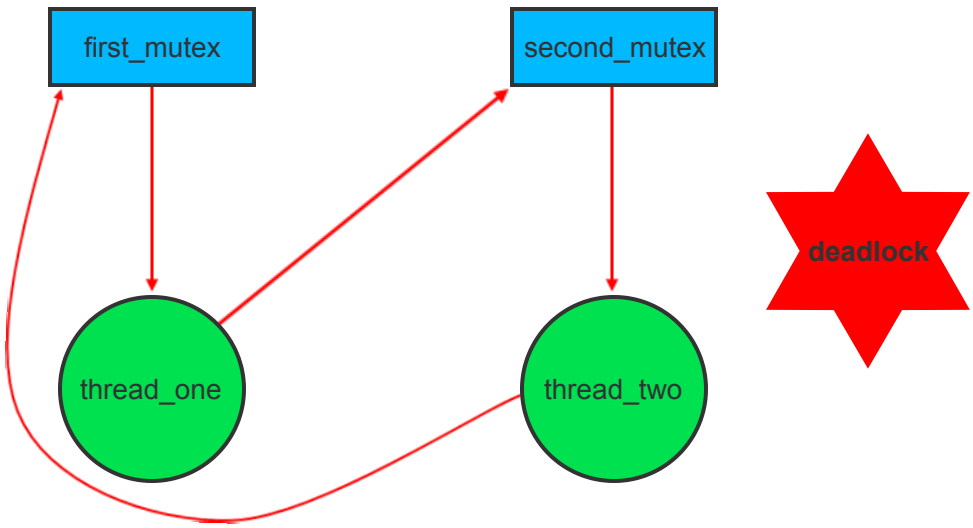
Resource-allocation graph

Deadlocks can be described more precisely in terms of a directed graph called a ***system resource-allocation graph***. This graph consists of a set of vertices V and a set of edges E. The set of vertices V is partitioned into two different types of nodes: $T = \{T_1, T_2, \ldots, T_n\}$, the set consisting of all the active threads in the system, and $R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system.

A directed edge from thread T_i to resource type R_j is denoted by $T_i \rightarrow R_j$; it signifies that thread T_i has requested an instance of resource type R_j and is currently waiting for that resource. A directed edge from resource type R_j to thread T_i is denoted by $R_j \rightarrow T_i$; it signifies that an instance of resource type R_j has been allocated to thread T_i. A directed edge $T_i \rightarrow R_j$ is called a ***request edge***; a directed edge $R_j \rightarrow T_i$ is called an ***assignment edge***.

Pictorially, we represent each thread T_i as a circle and each resource type R_j as a rectangle. As a simple example, the resource allocation graph shown in the animation below illustrates the deadlock situation from the program in Figure 5.9.1. Since resource type R_j may have more than one instance, we represent each such instance as a dot within the rectangle. Note that a request edge points only to the rectangle R_j, whereas an assignment edge must also designate one of the dots in the rectangle.

PARTICIPATION ACTIVITY 5.10.2: Resource-allocation graph for the program in Figure 8.3.1.



Animation content:

Step 1: thread_one acquires first_mutex. Step 2: thread_two acquires second_mutex. Step 3: thread_one wants second_mutex, which could be freed up by thread_two. Step 4: thread_two wants first_mutex, and as it is still holding second_mutex which thread_one is waiting for, there is a cycle in the graph: first_mutex → T1 → second_mutex → T2 → first_mutex.

Animation captions:

1. thread_one acquires first_mutex.
2. thread_two acquires second_mutex.

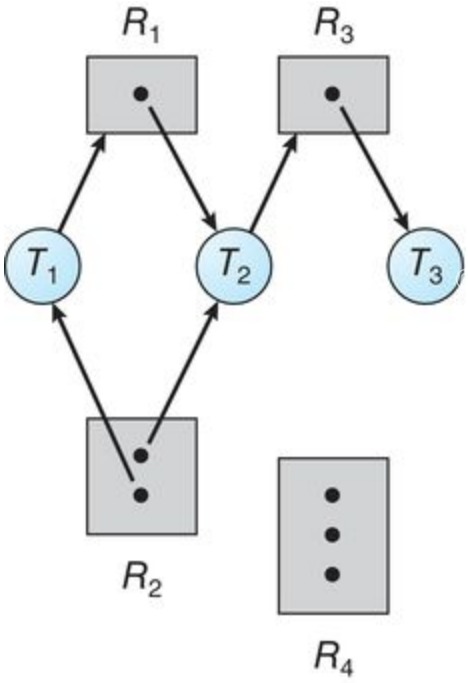
3. thread_one wants second_mutex, which could be freed up by thread_two.
4. thread_two wants first_mutex, and as it is still holding second_mutex which thread_one is waiting for, there is a cycle in the graph: first_mutex → T1 → second_mutex → T2 → first_mutex.

When thread T_i requests an instance of resource type R_j, a request edge is inserted in the resource-allocation graph. When this request can be fulfilled, the request edge is ***instantaneously*** transformed to an assignment edge. When the thread no longer needs access to the resource, it releases the resource. As a result, the assignment edge is deleted.

The resource-allocation graph shown in Figure 5.10.1 depicts the following situation.

- The sets T, R, and E:
  - $T = \{T_1, T_2, T_3\}$
  - $R = \{R_1, R_2, R_3, R_4\}$
  - $E = \{T_1 \rightarrow R_1, T_2 \rightarrow R_3, R_1 \rightarrow T_2, R_2 \rightarrow T_2, R_2 \rightarrow T_1, R_3 \rightarrow T_3\}$
- Resource instances:
  - One instance of resource type R_1
  - Two instances of resource type R_2
  - One instance of resource type R_3
  - Three instances of resource type R_4
- Thread states:
  - Thread T_1 is holding an instance of resource type R_2 and is waiting for an instance of resource type R_1.
  - Thread T_2 is holding an instance of R_1 and an instance of R_2 and is waiting for an instance of R_3.
  - Thread T_3 is holding an instance of R_3.

Figure 5.10.1: Resource-allocation graph.



Given the definition of a resource-allocation graph, it can be shown that, if the graph contains no cycles, then no thread in the system is deadlocked. If the graph does contain a cycle, then a deadlock may exist.

If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred. If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred. Each thread involved in the cycle is deadlocked. In this case, a cycle in the graph is both a necessary and a sufficient condition for the existence of deadlock.

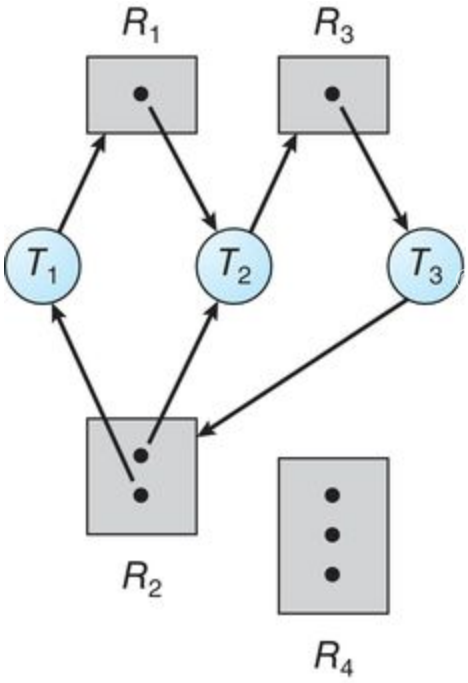
If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

To illustrate this concept, we return to the resource-allocation graph depicted in Figure 5.10.1. Suppose that thread T_3 requests an instance of resource type R_2. Since no resource instance is currently available, we add a request edge $T_3 \rightarrow R_2$ to the graph (Figure 5.10.2). At this point, two minimal cycles exist in the system:

```
T1 → R1 → T2 → R3 → T3 → R2 → T1
T2 → R3 → T3 → R2 → T2
```

Threads T_1, T_2, and T_3 are deadlocked. Thread T_2 is waiting for the resource R_3, which is held by thread T_3. Thread T_3 is waiting for either thread T_1 or thread T_2 to release resource R_2. In addition, thread T_1 is waiting for thread T_2 to release resource R_1.

Figure 5.10.2: Resource-allocation graph with a deadlock.

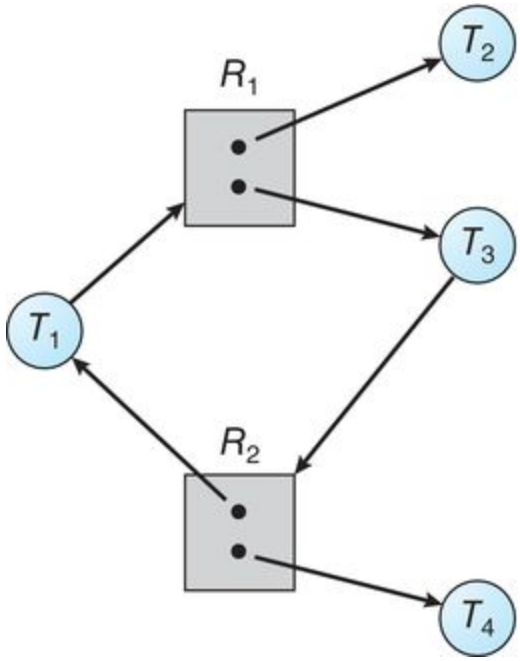


Now consider the resource-allocation graph in Figure 5.10.3. In this example, we also have a cycle:

```
T1 → R1 → T3 → R2 → T1
```

However, there is no deadlock. Observe that thread T_4 may release its instance of resource type R_2. That resource can then be allocated to T_3, breaking the cycle.

Figure 5.10.3: Resource-allocation graph with a cycle but no deadlock.



In summary, if a resource-allocation graph does not have a cycle, then the system is ***not*** in a deadlocked state. If there is a cycle, then the system ***may*** or ***may not*** be in a deadlocked state. This observation is important when we deal with the deadlock problem.

PARTICIPATION ACTIVITY 5.10.3: Section review question.

1) One necessary condition for deadlock is ____, which states that a process must be holding one resource and waiting to acquire additional resources.
   - hold and wait
   - mutual exclusion
   - no preemption

2) The presence of a cycle in a resource-allocation graph is ____.
   - a necessary and sufficient condition for deadlock in the case that each resource has more than one instance
   - a necessary and sufficient condition for a deadlock in the case that each resource has exactly one instance
   - is neither necessary nor sufficient for indicating deadlock in the case that each resource has exactly one instance

3) If a resource-allocation graph has a cycle, the system must be in a deadlock.
   - True
   - False

Section glossary

system resource-allocation graph: A directed graph for precise description of deadlocks.

request edge: In a system resource-allocation graph, an edge (arrow) indicating a resource request.

assignment edge: In a system resource-allocation graph, an edge (arrow) indicating a resource assignment.

5.11 Methods for handling deadlocks

Generally speaking, we can deal with the deadlock problem in one of three ways:

- We can ignore the problem altogether and pretend that deadlocks never occur in the system.
- We can use a protocol to prevent or avoid deadlocks, ensuring that the system will ***never*** enter a deadlocked state.
- We can allow the system to enter a deadlocked state, detect it, and recover.

The first solution is the one used by most operating systems, including Linux and Windows. It is then up to kernel and application developers to write programs that handle deadlocks, typically using approaches outlined in the second solution. Some systems—such as databases—adopt the third solution, allowing deadlocks to occur and then managing the recovery.

Next, we elaborate briefly on the three methods for handling deadlocks. Then, in Section 5.12 through Section Missing Reference : Recovery from deadlock, we present detailed algorithms. Before proceeding, we should mention that some researchers have argued that none of the basic approaches alone is appropriate for the entire spectrum of resource-allocation problems in operating systems. The basic approaches can be combined, however, allowing us to select an optimal approach for each class of resources in a system.

To ensure that deadlocks never occur, the system can use either a deadlock-prevention or a deadlock-avoidance scheme. ***Deadlock prevention*** provides a set of methods to ensure that at least one of the necessary conditions (Section Necessary conditions) cannot hold. These methods prevent deadlocks by constraining how requests for resources can be made. We discuss these methods in Section 5.12.

Deadlock avoidance requires that the operating system be given additional information in advance concerning which resources a thread will request and use during its lifetime. With this additional knowledge, the operating system can decide for each request whether or not the thread should wait. To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each thread, and the future requests and releases of each thread. We discuss these schemes in Section 5.13.

If a system does not employ either a deadlock-prevention or a deadlock-avoidance algorithm, then a deadlock situation may arise. In this environment, the system can provide an algorithm that examines the state of the system to determine whether a deadlock has occurred and an algorithm to recover from the deadlock (if a deadlock has indeed occurred). We discuss these issues in Section 5.14 and Section Missing Reference : Recovery from deadlock.

In the absence of algorithms to detect and recover from deadlocks, we may arrive at a situation in which the system is in a deadlocked state yet has no way of recognizing what has happened. In this case, the undetected deadlock will cause the system's performance to deteriorate, because resources are being held by threads that cannot run and because more and more threads, as they make requests for resources, will enter a deadlocked state. Eventually, the system will stop functioning and will need to be restarted manually.

Although this method may not seem to be a viable approach to the deadlock problem, it is nevertheless used in most operating systems, as mentioned earlier. Expense is one important consideration. Ignoring the possibility of deadlocks is cheaper than the other approaches. Since in many systems, deadlocks occur infrequently (say, once per month), the extra expense of the other methods may not seem worthwhile.

In addition, methods used to recover from other liveness conditions, such as livelock, may be used to recover from deadlock. In some circumstances, a system is suffering from a liveness failure but is not in a deadlocked state. We see this situation, for example, with a real-time thread running at the highest priority (or any thread running on a nonpreemptive scheduler) and never returning control to the operating system. The system must have manual recovery methods for such conditions and may simply use those techniques for deadlock recovery.

PARTICIPATION ACTIVITY 5.11.1: Section review questions.

1) To handle deadlocks, operating systems most often ____.
   - ignore the problem of deadlocks
   - use protocols to prevent or avoid deadlocks
   - detect and recover from deadlocks

2) Deadlock _____ handles deadlocks by ensuring that one of their necessary conditions cannot hold.
   - avoidance
   - prevention
   - recovery

Section glossary

deadlock prevention: A set of methods intended to ensure that at least one of the necessary conditions for deadlock cannot hold.

deadlock avoidance: An operating system method in which processes inform the operating system of which resources they will use during their lifetimes so the system can approve or deny requests to avoid deadlock.

5.12 Deadlock prevention

As we noted in Section Necessary conditions, for a deadlock to occur, each of the four necessary conditions must hold. By ensuring that at least one of these conditions cannot hold, we can ***prevent*** the occurrence of a deadlock. We elaborate on this approach by examining each of the four necessary conditions separately.

Mutual exclusion

The mutual-exclusion condition must hold. That is, at least one resource must be nonsharable. Sharable resources do not require mutually exclusive access and thus cannot be involved in a deadlock. Read-only files are a good example of a sharable resource. If several threads attempt to open a read-only file at the same time, they can be granted simultaneous access to the file. A thread never needs to wait for a sharable resource. In general, however, we cannot prevent deadlocks by denying the mutual-exclusion condition, because some resources are intrinsically nonsharable. For example, a mutex lock cannot be simultaneously shared by several threads.

Hold and wait

To ensure that the hold-and-wait condition never occurs in the system, we must guarantee that, whenever a thread requests a resource, it does not hold any other resources. One protocol that we can use requires each thread to request and be allocated all its resources before it begins execution. This is, of course, impractical for most applications due to the dynamic nature of requesting resources.

An alternative protocol allows a thread to request resources only when it has none. A thread may request some resources and use them. Before it can request any additional resources, it must release all the resources that it is currently allocated.

Both these protocols have two main disadvantages. First, resource utilization may be low, since resources may be allocated but unused for a long period. For example, a thread may be allocated a mutex lock for its entire execution, yet only require it for a short duration. Second, starvation is possible. A thread that needs several popular resources may have to wait indefinitely, because at least one of the resources that it needs is always allocated to some other thread.

No preemption

The third necessary condition for deadlocks is that there be no preemption of resources that have already been allocated. To ensure that this condition does not hold, we can use the following protocol. If a thread is holding some resources and requests another resource that cannot be immediately allocated to it (that is, the thread must wait), then all resources the thread is currently holding are preempted. In other words, these resources are implicitly released. The preempted resources are added to the list of resources for which the thread is waiting. The thread will be restarted only when it can regain its old resources, as well as the new ones that it is requesting.

Alternatively, if a thread requests some resources, we first check whether they are available. If they are, we allocate them. If they are not, we check whether they are allocated to some other thread that is waiting for additional resources. If so, we preempt the desired resources from the waiting thread and allocate them to the requesting thread. If the resources are neither available nor held by a waiting thread, the requesting thread must wait. While it is waiting, some of its resources may be preempted, but only if another thread requests them. A thread can be restarted only when it is allocated the new resources it is requesting and recovers any resources that were preempted while it was waiting.

This protocol is often applied to resources whose state can be easily saved and restored later, such as CPU registers and database transactions. It cannot generally be applied to such resources as mutex locks and semaphores, precisely the type of resources where deadlock occurs most commonly.

Circular wait

The three options presented thus far for deadlock prevention are generally impractical in most situations. However, the fourth and final condition for deadlocks — the circular-wait condition — presents an opportunity for a practical solution by invalidating one of the necessary conditions. One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each thread requests resources in an increasing order of enumeration.

To illustrate, we let $\boldsymbol{R}$ = {$\boldsymbol{R_1}$, $\boldsymbol{R_2}$, ..., $\boldsymbol{R_m}$} be the set of resource types. We assign to each resource type a unique integer number, which allows us to compare two resources and to determine whether one precedes another in our ordering. Formally, we define a one-to-one function $\boldsymbol{F}$: $\boldsymbol{R \rightarrow N}$, where $\boldsymbol{N}$ is the set of natural numbers. We can accomplish this scheme in an application program by developing an ordering among all synchronization objects in the system. For example, the lock ordering in the Pthread program shown in Figure 5.9.1 could be

```
F(first_mutex) = 1
F(second_mutex) = 5
```


We can now consider the following protocol to prevent deadlocks: Each thread can request resources only in an increasing order of enumeration. That is, a thread can initially request an instance of a resource—say, $\boldsymbol{R_i}$. After that, the thread can request an instance of resource $\boldsymbol{R_j}$ if and only if $\boldsymbol{F(R_j) > F(R_i)}$. For example, using the function defined above, a thread that wants to use both `first_mutex` and `second_mutex` at the same time must first request `first_mutex` and then `second_mutex`.

Alternatively, we can require that a thread requesting an instance of resource R_j must have released any resources R_i such that $F(R_i) \geq F(R_j)$. Note also that if several instances of the same resource type are needed, a ***single*** request for all of them must be issued.

If these two protocols are used, then the circular-wait condition cannot hold. We can demonstrate this fact by assuming that a circular wait exists (proof by contradiction). Let the set of threads involved in the circular wait be $\{T_0, T_1, \ldots, T_n\}$, where T_i is waiting for a resource R_i, which is held by thread T_{i+1}. (Modulo arithmetic is used on the indexes, so that T_n is waiting for a resource R_n held by T_0.) Then, since thread T_{i+1} is holding resource R_i while requesting resource R_{i+1}, we must have $F(R_i) < F(R_{i+1})$ for all i. But this condition means that $F(R_0) < F(R_1) < \ldots < F(R_n) < F(R_0)$. By transitivity, $F(R_0) < F(R_0)$, which is impossible. Therefore, there can be no circular wait.

Keep in mind that developing an ordering, or hierarchy, does not in itself prevent deadlock. It is up to application developers to write programs that follow the ordering. However, establishing a lock ordering can be difficult, especially on a system with hundreds—or thousands—of locks. To address this challenge, many Java developers have adopted the strategy of using the method `System.identityHashCode(Object)` (which returns the default hash code value of the `Object` parameter it has been passed) as the function for ordering lock acquisition.

It is also important to note that imposing a lock ordering does not guarantee deadlock prevention if locks can be acquired dynamically. For example, assume we have a function that transfers funds between two accounts. To prevent a race condition, each account has an associated mutex lock that is obtained from a `get_lock()` function such as that shown in Figure 5.12.1. Deadlock is possible if two threads simultaneously invoke the `transaction()` function, transposing different accounts. That is, one thread might invoke

```
transaction(checking_account, savings_account, 25.0)
```

and another might invoke

```
transaction(savings_account, checking_account, 50.0)
```

Figure 5.12.1: Deadlock example with lock ordering.

```
void transaction(Account from, Account to, double amount)
{
  mutex lock1, lock2;
  lock1 = get_lock(from);
  lock2 = get_lock(to);

  acquire(lock1);
    acquire(lock2);

      withdraw(from, amount);
      deposit(to, amount);

    release(lock2);
```

```
  release(lock1);
}
```

PARTICIPATION ACTIVITY 5.12.1: Section review question.

1) Deadlock prevention by denying the _____ condition is the simplest way to prevent deadlocks.
   - mutual exclusion
   - no preemption
   - circular wait

2) Assume there are three resources, R_1, R_2, and R_3, that are each assigned unique integer values 15, 10, and 25, respectively. What is a resource ordering which prevents a circular wait?
   - R_1, R_2, R_3
   - R_2, R_1, R_3
   - R_3, R_1, R_2

5.13 Deadlock avoidance

Deadlock-prevention algorithms, as discussed in Section 5.12, prevent deadlocks by limiting how requests can be made. The limits ensure that at least one of the necessary conditions for deadlock cannot occur. Possible side effects of preventing deadlocks by this method, however, are low device utilization and reduced system throughput.

An alternative method for avoiding deadlocks is to require additional information about how resources are to be requested. For example, in a system with resources *R*1 and *R*2, the system might need to know that thread P will request first *R*1 and then *R*2 before releasing both resources, whereas thread Q will request *R*2 and then *R*1. With this knowledge of the complete sequence of requests and releases for each thread, the system can decide for each request whether or not the thread should wait in order to avoid a possible future deadlock. Each request requires that in making this decision the system consider the resources currently available, the resources currently allocated to each thread, and the future requests and releases of each thread.

The various algorithms that use this approach differ in the amount and type of information required. The simplest and most useful model requires that each thread declare the ***maximum number*** of resources of each type that it may need. Given this a priori information, it is possible to construct an algorithm that ensures that the system will never enter a deadlocked state. A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular-wait condition can never exist. The resource-allocation ***state*** is defined by the number of available and allocated resources and the maximum demands of the threads. In the following sections, we explore two deadlock-avoidance algorithms.

Linux lockdep tool

Although ensuring that resources are acquired in the proper order is the responsibility of kernel and application developers, certain software can be used to verify that locks are acquired in the proper order. To detect possible deadlocks, Linux provides `lockdep`, a tool with rich functionality that can be used to verify locking order in the kernel. `lockdep` is designed to be enabled on a running kernel as it monitors usage patterns of lock acquisitions and releases against a set of rules for acquiring and releasing locks. Two examples follow, but note that `lockdep` provides significantly more functionality than what is described here:

- The order in which locks are acquired is dynamically maintained by the system. If `lockdep` detects locks being acquired out of order, it reports a possible deadlock condition.
- In Linux, spinlocks can be used in interrupt handlers. A possible source of deadlock occurs when the kernel acquires a spinlock that is also used in an interrupt handler. If the interrupt occurs while the lock is being held, the interrupt handler preempts the kernel code currently holding the lock and then spins while attempting to acquire the lock, resulting in deadlock. The general strategy for avoiding this situation is to disable interrupts on the current processor before acquiring a spinlock that is also used in an interrupt handler. If `lockdep` detects that interrupts are enabled while kernel code acquires a lock that is also used in an interrupt handler, it will report a possible deadlock scenario.

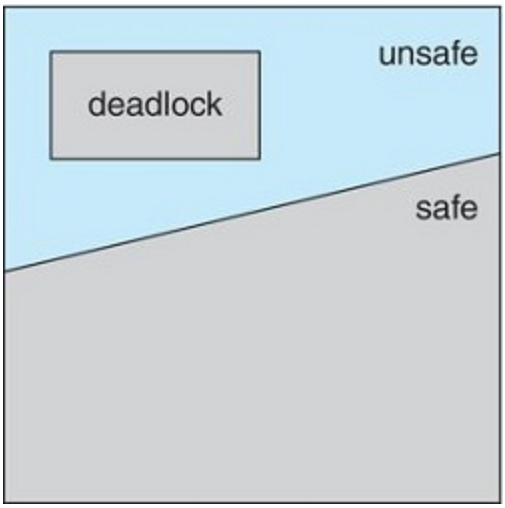
`lockdep` was developed to be used as a tool in developing or modifying code in the kernel and not to be used on production systems, as it can significantly slow down a system. Its purpose is to test whether software such as a new device driver or kernel module provides a possible source of deadlock. The designers of `lockdep` have reported that within a few years of its development in 2006, the number of deadlocks from system reports had been reduced by an order of magnitude. Although `lockdep` was originally designed only for use in the kernel, recent revisions of this tool can now be used for detecting deadlocks in user applications using Pthreads mutex locks. Further details on the `lockdep` tool can be found at https://www.kernel.org/doc/Documentation/locking/lockdep-design.txt.

Safe state

A state is *safe* if the system can allocate resources to each thread (up to its maximum) in some order and still avoid a deadlock. More formally, a system is in a safe state only if there exists a ***safe sequence***. A sequence of threads <T_1, T_2, ..., T_n> is a safe sequence for the current allocation state if, for each T_i, the resource requests that T_i can still make can be satisfied by the currently available resources plus the resources held by all T_j, with $j < i$. In this situation, if the resources that T_i needs are not immediately available, then T_i can wait until all T_j have finished. When they have finished, T_i can obtain all of its needed resources, complete its designated task, return its allocated resources, and terminate. When T_i terminates, T_{i+1} can obtain its needed resources, and so on. If no such sequence exists, then the system state is said to be *unsafe*.

A safe state is not a deadlocked state. Conversely, a deadlocked state is an unsafe state. Not all unsafe states are deadlocks, however (Figure 5.13.1). An unsafe state ***may*** lead to a deadlock. As long as the state is safe, the operating system can avoid unsafe (and deadlocked) states. In an unsafe state, the operating system cannot prevent threads from requesting resources in such a way that a deadlock occurs. The behavior of the threads controls unsafe states.

Figure 5.13.1: Safe, unsafe, and deadlocked state spaces.



PARTICIPATION ACTIVITY 5.13.1: Mid-section review question.

Only an unsafe state can lead to deadlock.

- True
- False

To illustrate, consider a system with twelve resources and three threads: T_0, T_1, and T_2. Thread T_0 requires ten resources, thread T_1 may need as many as four, and thread T_2 may need up to nine

resources. Suppose that, at time t_0, thread T_0 is holding five resources, thread T_1 is holding two resources, and thread T_2 is holding two resources. (Thus, there are three free resources.)

	Maximum Needs	Current Needs
T_0	10	5
T_1	4	2
T_2	9	2

At time t_0, the system is in a safe state. The sequence <T_1, T_0, T_2> satisfies the safety condition. Thread T_1 can immediately be allocated all its resources and then return them (the system will then have five available resources); then thread T_0 can get all its resources and return them (the system will then have ten available resources); and finally thread T_2 can get all its resources and return them (the system will then have all twelve resources available).

A system can go from a safe state to an unsafe state. Suppose that, at time t_1, thread T_2 requests and is allocated one more resource. The system is no longer in a safe state. At this point, only thread T_1 can be allocated all its resources. When it returns them, the system will have only four available resources. Since thread T_0 is allocated five resources but has a maximum of ten, it may request five more resources. If it does so, it will have to wait, because they are unavailable. Similarly, thread T_2 may request six additional resources and have to wait, resulting in a deadlock. Our mistake was in granting the request from thread T_2 for one more resource. If we had made T_2 wait until either of the other threads had finished and released its resources, then we could have avoided the deadlock.

Given the concept of a safe state, we can define avoidance algorithms that ensure that the system will never deadlock. The idea is simply to ensure that the system will always remain in a safe state. Initially, the system is in a safe state. Whenever a thread requests a resource that is currently available, the system must decide whether the resource can be allocated immediately or the thread must wait. The request is granted only if the allocation leaves the system in a safe state.

In this scheme, if a thread requests a resource that is currently available, it may still have to wait. Thus, resource utilization may be lower than it would otherwise be.

Resource-allocation-graph algorithm

If we have a resource-allocation system with only one instance of each resource type, we can use a variant of the resource-allocation graph defined in Section Resource-allocation graph for deadlock avoidance. In addition to the request and assignment edges already described, we introduce a new type of edge, called a ***claim edge***. A claim edge $T_i \rightarrow R_j$ indicates that thread T_i may request resource R_j at some time in the future. This edge resembles a request edge in direction but is represented in the graph by a dashed line. When thread T_i requests resource R_j, the claim edge $T_i \rightarrow R_j$ is converted to a request edge. Similarly,

when a resource R_j is released by T_i, the assignment edge $R_j \rightarrow T_i$ is reconverted to a claim edge $T_i \rightarrow R_j$.

Note that the resources must be claimed a priori in the system. That is, before thread T_i starts executing, all its claim edges must already appear in the resource-allocation graph. We can relax this condition by allowing a claim edge $T_i \rightarrow R_j$ to be added to the graph only if all the edges associated with thread T_i are claim edges.

Now suppose that thread T_i requests resource R_j. The request can be granted only if converting the request edge $T_i \rightarrow R_j$ to an assignment edge $R_j \rightarrow T_i$ does not result in the formation of a cycle in the resource-allocation graph. We check for safety by using a cycle-detection algorithm. An algorithm for detecting a cycle in this graph requires an order of n^2 operations, where n is the number of threads in the system.

If no cycle exists, then the allocation of the resource will leave the system in a safe state. If a cycle is found, then the allocation will put the system in an unsafe state. In that case, thread T_i will have to wait for its requests to be satisfied.

To illustrate this algorithm, we consider the resource-allocation graph of Figure 5.13.2. Suppose that T_2 requests R_2. Although R_2 is currently free, we cannot allocate it to T_2, since this action will create a cycle in the graph (Figure 5.13.3). A cycle, as mentioned, indicates that the system is in an unsafe state. If T_1 requests R_2, and T_2 requests R_1, then a deadlock will occur.

Figure 5.13.2: Resource-allocation graph for deadlock avoidance.

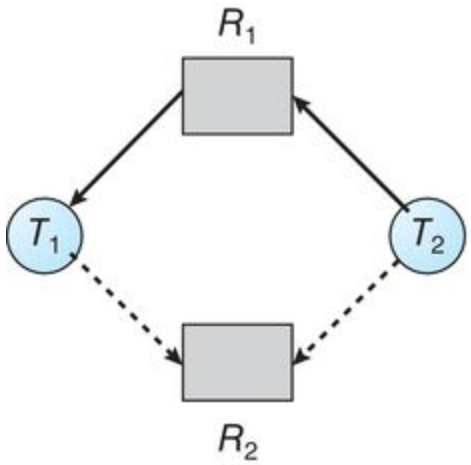
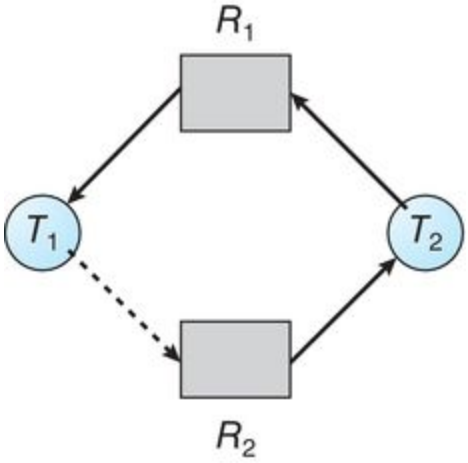


Figure 5.13.3: An unsafe state in a resource-allocation graph.



Banker's algorithm

The resource-allocation-graph algorithm is not applicable to a resource-allocation system with multiple instances of each resource type. The deadlock-avoidance algorithm that we describe next is applicable to such a system but is less efficient than the resource-allocation graph scheme. This algorithm is commonly known as the ***banker's algorithm***. The name was chosen because the algorithm could be used in a banking system to ensure that the bank never allocated its available cash in such a way that it could no longer satisfy the needs of all its customers.

When a new thread enters the system, it must declare the maximum number of instances of each resource type that it may need. This number may not exceed the total number of resources in the system. When a user requests a set of resources, the system must determine whether the allocation of these resources will leave the system in a safe state. If it will, the resources are allocated; otherwise, the thread must wait until some other thread releases enough resources.

Several data structures must be maintained to implement the banker's algorithm. These data structures encode the state of the resource-allocation system. We need the following data structures, where n is the number of threads in the system and m is the number of resource types:

- **Available.** A vector of length m indicates the number of available resources of each type. If ***Available***[j] equals k, then k instances of resource type R_j are available.
- **Max.** An $n \times m$ matrix defines the maximum demand of each thread. If ***Max***[i][j] equals k, then thread T_i may request at most k instances of resource type R_j.
- **Allocation.** An $n \times m$ matrix defines the number of resources of each type currently allocated to each thread. If ***Allocation***[i][j] equals k, then thread T_i is currently allocated k instances of resource type R_j.
- **Need.** An $n \times m$ matrix indicates the remaining resource need of each thread. If ***Need***[i][j] equals k, then thread T_i may need k more instances of resource type R_j to complete its task. Note that ***Need***[i][j] equals ***Max***[i][j] - ***Allocation***[i][j].

These data structures vary over time in both size and value.

To simplify the presentation of the banker's algorithm, we next establish some notation. Let X and Y be vectors of length n. We say that $X \leq Y$ if and only if $X[i] \leq Y[i]$ for all i = 1, 2, …, n. For example, if X = (1,7,3,2) and Y = (0,3,2,1), then $Y \leq X$. In addition, $Y < X$ if $Y \leq X$ and $Y \neq X$.

We can treat each row in the matrices ***Allocation*** and ***Need*** as vectors and refer to them as ***Allocation***$_i$ and ***Need***$_i$. The vector ***Allocation***$_i$ specifies the resources currently allocated to thread T_i; the vector ***Need***$_i$ specifies the additional resources that thread T_i may still request to complete its task.

Safety algorithm

We can now present the algorithm for finding out whether or not a system is in a safe state. This algorithm can be described as follows:

1. Let ***Work*** and ***Finish*** be vectors of length m and n, respectively. Initialize ***Work*** = ***Available*** and ***Finish***[i] = ***false*** for i = 0, 1, …, n − 1.
2. Find an index i such that both

   a. ***Finish***[i] == ***false***

   b. ***Need***$_i \leq$ ***Work***

   If no such i exists, go to step 4.
3. ***Work*** = ***Work*** + ***Allocation***$_i$ ***Finish***[i] = ***true*** Go to step 2.
4. If ***Finish***[i] == ***true*** for all i, then the system is in a safe state.

This algorithm may require an order of $m \times n^2$ operations to determine whether a state is safe.

Resource-request algorithm

Next, we describe the algorithm for determining whether requests can be safely granted. Let ***Request***$_i$ be the request vector for thread T_i. If ***Request***$_i$ [j] == k, then thread T_i wants k instances of resource type R_j. When a request for resources is made by thread T_i, the following actions are taken:

1. If ***Request***$_i \leq$ ***Need***$_i$, go to step 2. Otherwise, raise an error condition, since the thread has exceeded its maximum claim.
2. If ***Request***$_i \leq$ ***Available***, go to step 3. Otherwise, T_i must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to thread T_i by modifying the state as follows:

   Available = ***Available*** - ***Request***$_i$
   Allocation$_i$ = ***Allocation***$_i$ + ***Request***$_i$
   Need$_i$ = ***Need***$_i$ - ***Request***$_i$

If the resulting resource-allocation state is safe, the transaction is completed, and thread T_i is allocated its resources. However, if the new state is unsafe, then T_i must wait for ***Request***$_i$, and the old resource-allocation state is restored.

An illustrative example

To illustrate the use of the banker's algorithm, consider a system with five threads T_0 through T_4 and three resource types A, B, and C. Resource type A has ten instances, resource type B has five instances, and resource type C has seven instances. Suppose that the following snapshot represents the current state of the system:

	Allocation	Max	Available
	A B C	*A B C*	*A B C*
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

The content of the matrix ***Need*** is defined to be ***Max*** − ***Allocation*** and is as follows:

	Need
	A B C
T_0	7 4 3
T_1	1 2 2
T_2	6 0 0
T_3	0 1 1
T_4	4 3 1

We claim that the system is currently in a safe state. Indeed, the sequence <T_1, T_3, T_4, T_2, T_0> satisfies the safety criteria. Suppose now that thread T_1 requests one additional instance of resource type A and

two instances of resource type C, so ***Request***$_1$ = (1,0,2). To decide whether this request can be immediately granted, we first check that ***Request***$_1$ ≤ ***Available***—that is, that (1,0,2) ≤ (3,3,2), which is true. We then pretend that this request has been fulfilled, and we arrive at the following new state:

	Allocation	Need	Available
	A B C	*A B C*	*A B C*
T_0	0 1 0	7 4 3	2 3 0
T_1	3 0 2	0 2 0	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

We must determine whether this new system state is safe. To do so, we execute our safety algorithm and find that the sequence <T_1, T_3, T_4, T_0, T_2> satisfies the safety requirement. Hence, we can immediately grant the request of thread T_1.

You should be able to see, however, that when the system is in this state, a request for (3,3,0) by T_4 cannot be granted, since the resources are not available. Furthermore, a request for (0,2,0) by T_0 cannot be granted, even though the resources are available, since the resulting state is unsafe: granting this request would result in ***Available*** = (2, 1, 0) which is less than the value of ***Need*** for all threads.

We leave it as a programming exercise for students to implement the banker's algorithm.

PARTICIPATION ACTIVITY 5.13.2: Section review questions.

1) Suppose that there are ten resources available to three processes. At time 0, the following data is collected. The table indicates the process, the maximum number of resources needed by the process, and the number of resources currently owned by each process. Which of the following correctly characterizes this state?

Process	Maximum Needs	Currently Owned
P_0	10	4
P_1	3	1
P_2	6	4

- It is safe.
- It is not safe.
- The state cannot be determined.

2) Suppose that there are 12 resources available to three processes. At time 0, the following data is collected. The table indicates the process, the maximum number of resources needed by the process, and the number of resources currently owned by each process. Which of the following correctly characterizes this state?

Process	Maximum Needs	Currently Owned
P_0	10	4
P_1	3	2
P_2	7	4

- It is safe.
- It is not safe.
- The state cannot be determined.

Section glossary

safe sequence: In deadlock avoidance, a sequence of processes <P1, P2, ..., Pn> in which, for each Pi, the resource requests that Pi can make can be satisfied by the currently available resources plus the resources held by all Pj, with j < i.

claim edge: In the deadlock resource-allocation-graph algorithm, an edge indicating that a process might claim a resource in the future.

banker's algorithm: A deadlock avoidance algorithm, less efficient than the resource-allocation graph scheme but able to deal with multiple instances of each resource type.

5.14 Deadlock detection

If a system does not employ either a deadlock-prevention or a deadlock-avoidance algorithm, then a deadlock situation may occur. In this environment, the system may provide:

- An algorithm that examines the state of the system to determine whether a deadlock has occurred
- An algorithm to recover from the deadlock

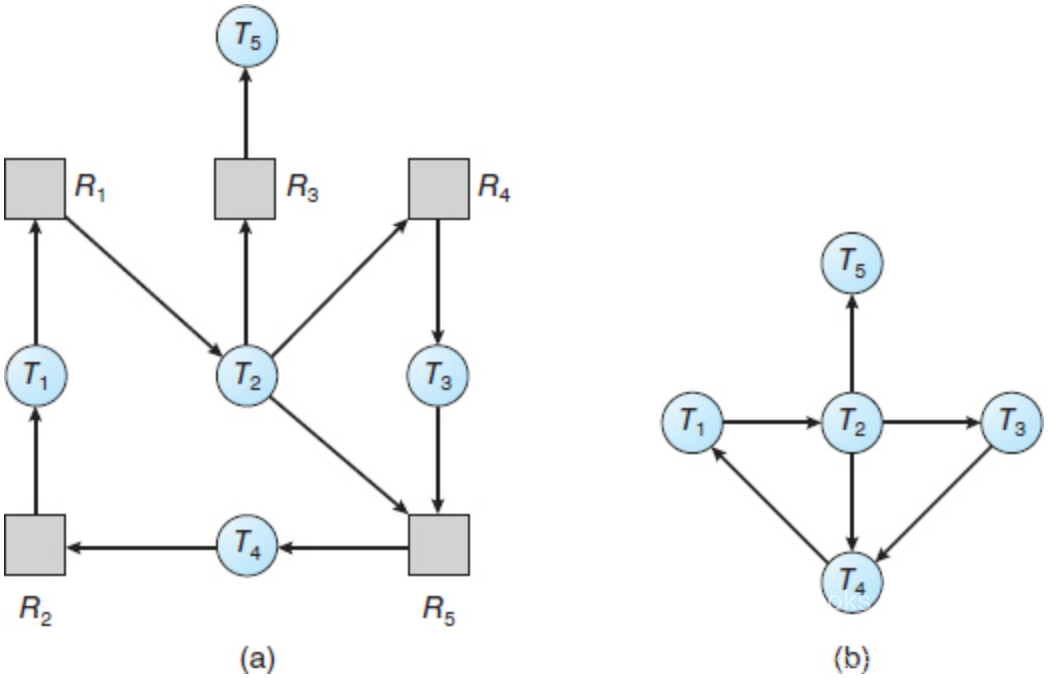
Next, we discuss these two requirements as they pertain to systems with only a single instance of each resource type, as well as to systems with several instances of each resource type. At this point, however, we note that a detection-and-recovery scheme requires overhead that includes not only the run-time costs of maintaining the necessary information and executing the detection algorithm but also the potential losses inherent in recovering from a deadlock.

Single instance of each resource type

If all resources have only a single instance, then we can define a deadlock-detection algorithm that uses a variant of the resource-allocation graph, called a ***wait-for*** graph. We obtain this graph from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges.

More precisely, an edge from T_i to T_j in a wait-for graph implies that thread T_i is waiting for thread T_j to release a resource that T_i needs. An edge $T_i \rightarrow T_j$ exists in a wait-for graph if and only if the corresponding resource-allocation graph contains two edges $T_i \rightarrow R_q$ and $R_q \rightarrow T_j$ for some resource R_q. In Figure 5.14.1, we present a resource-allocation graph and the corresponding wait-for graph.

Figure 5.14.1: (a) Resource-allocation graph. (b) Corresponding wait-for graph.



As before, a deadlock exists in the system if and only if the wait-for graph contains a cycle. To detect deadlocks, the system needs to ***maintain*** the wait-for graph and periodically ***invoke an algorithm*** that searches for a cycle in the graph. An algorithm to detect a cycle in a graph requires $O(n^2)$ operations, where n is the number of vertices in the graph.

The BCC toolkit described in Section BCC provides a tool that can detect potential deadlocks with Pthreads mutex locks in a user process running on a Linux system. The BCC tool `deadlock_detector` operates by inserting probes which trace calls to the `pthread_mutex_lock()` and `pthread_mutex_unlock()` functions. When the specified process makes a call to either function, `deadlock_detector` constructs a wait-for graph of mutex locks in that process, and reports the possibility of deadlock if it detects a cycle in the graph.

Several instances of a resource type

The wait-for graph scheme is not applicable to a resource-allocation system with multiple instances of each resource type. We turn now to a deadlock-detection algorithm that is applicable to such a system. The algorithm employs several time-varying data structures that are similar to those used in the banker's algorithm (Section Banker's algorithm):

- **Available.** A vector of length m indicates the number of available resources of each type.
- **Allocation.** An $n \times m$ matrix defines the number of resources of each type currently allocated to each thread.
- **Request.** An $n \times m$ matrix indicates the current request of each thread. If ***Request***$[i][j]$ equals k, then thread T_i is requesting k more instances of resource type R_j.

Deadlock detection using java thread dumps

Although Java does not provide explicit support for deadlock detection, a ***thread dump*** can be used to analyze a running program to determine if there is a deadlock. A thread dump is a useful debugging tool that displays a snapshot of the states of all threads in a Java application. Java thread dumps also show locking information, including which locks a blocked thread is waiting to acquire. When a thread dump is generated, the JVM searches the wait-for graph to detect cycles, reporting any deadlocks it detects. To generate a thread dump of a running application, from the command line enter:

```
Ctrl-\ (UNIX, Linux, or macOS)
Ctrl-Break (Windows)
```

In the source-code download for this text, we provide a Java example of the program shown in Figure 5.9.1 and describe how to generate a thread dump that reports the deadlocked Java threads.

The ≤ relation between two vectors is defined as in Section Banker's algorithm. To simplify notation, we again treat the rows in the matrices ***Allocation*** and ***Request*** as vectors; we refer to them as ***Allocation***$_i$ and ***Request***$_i$. The detection algorithm described here simply investigates every possible allocation sequence

for the threads that remain to be completed. Compare this algorithm with the banker's algorithm of Section Banker's algorithm.

1. Let ***Work*** and ***Finish*** be vectors of length m and n, respectively. Initialize ***Work*** = ***Available***. For i = 0, 1, ..., n−1, if ***Allocation***$_i$ ≠ 0, then ***Finish***[i] = ***false***. Otherwise, ***Finish***[i] = ***true***.
2. Find an index i such that both
   a. ***Finish***[i] == ***false***
   b. ***Request***$_i$ ≤ ***Work***

   If no such i exists, go to step 4.
3. ***Work*** = ***Work*** + ***Allocation***$_i$ ***Finish***[i] = ***true*** Go to step 2.
4. If ***Finish***[i] == ***false*** for some i, 0 ≤ i < n, then the system is in a deadlocked state. Moreover, if ***Finish***[i] == ***false***, then thread T_i is deadlocked.

This algorithm requires an order of $m \times n^2$ operations to detect whether the system is in a deadlocked state.

You may wonder why we reclaim the resources of thread T_i (in step 3) as soon as we determine that ***Request***$_i$ ≤ ***Work*** (in step 2b). We know that T_i is currently ***not*** involved in a deadlock (since ***Request***$_i$ ≤ ***Work***). Thus, we take an optimistic attitude and assume that T_i will require no more resources to complete its task; it will thus soon return all currently allocated resources to the system. If our assumption is incorrect, a deadlock may occur later. That deadlock will be detected the next time the deadlock-detection algorithm is invoked.

To illustrate this algorithm, we consider a system with five threads T_0 through T_4 and three resource types A, B, and C. Resource type A has seven instances, resource type B has two instances, and resource type C has six instances. The following snapshot represents the current state of the system:

	Allocation	Request	Available
	A B C	*A B C*	*A B C*
T_0	0 1 0	0 0 0	0 0 0
T_1	2 0 0	2 0 2	
T_2	3 0 3	0 0 0	
T_3	2 1 1	1 0 0	
T_4	0 0 2	0 0 2	

We claim that the system is not in a deadlocked state. Indeed, if we execute our algorithm, we will find that the sequence <T_0, T_2, T_3, T_1, T_4> results in ***Finish***[i] == *true* for all i.

Suppose now that thread T_2 makes one additional request for an instance of type C. The ***Request*** matrix is modified as follows:

	Request
	A B C
T_0	0 0 0
T_1	2 0 2
T_2	0 0 1
T_3	1 0 0
T_4	0 0 2

We claim that the system is now deadlocked. Although we can reclaim the resources held by thread T_0, the number of available resources is not sufficient to fulfill the requests of the other threads. Thus, a deadlock exists, consisting of threads T_1, T_2, T_3, and T_4.

Detection-algorithm usage

When should we invoke the detection algorithm? The answer depends on two factors:

1. How ***often*** is a deadlock likely to occur?
2. How ***many*** threads will be affected by deadlock when it happens?

If deadlocks occur frequently, then the detection algorithm should be invoked frequently. Resources allocated to deadlocked threads will be idle until the deadlock can be broken. In addition, the number of threads involved in the deadlock cycle may grow.

Managing deadlock in databases

Database systems provide a useful illustration of how both open-source and commercial software manage deadlock. Updates to a database may be performed as ***transactions***, and to ensure data integrity, locks are typically used. A transaction may involve several locks, so it comes as no surprise that deadlocks are possible in a database with multiple concurrent transactions. To manage deadlock, most transactional database systems include a deadlock detection and recovery mechanism. The database server will periodically search for cycles in the wait-for graph to detect deadlock among a set of transactions. When deadlock is detected, a

victim is selected and the transaction is aborted and rolled back, releasing the locks held by the victim transaction and freeing the remaining transactions from deadlock. Once the remaining transactions have resumed, the aborted transaction is reissued. Choice of a victim transaction depends on the database system; for instance, MySQL attempts to select transactions that minimize the number of rows being inserted, updated, or deleted.

Deadlocks occur only when some thread makes a request that cannot be granted immediately. This request may be the final request that completes a chain of waiting threads. In the extreme, then, we can invoke the deadlock-detection algorithm every time a request for allocation cannot be granted immediately. In this case, we can identify not only the deadlocked set of threads but also the specific thread that "caused" the deadlock. (In reality, each of the deadlocked threads is a link in the cycle in the resource graph, so all of them, jointly, caused the deadlock.) If there are many different resource types, one request may create many cycles in the resource graph, each cycle completed by the most recent request and "caused" by the one identifiable thread.

Of course, invoking the deadlock-detection algorithm for every resource request will incur considerable overhead in computation time. A less expensive alternative is simply to invoke the algorithm at defined intervals—for example, once per hour or whenever CPU utilization drops below 40 percent. (A deadlock eventually cripples system throughput and causes CPU utilization to drop.) If the detection algorithm is invoked at arbitrary points in time, the resource graph may contain many cycles. In this case, we generally cannot tell which of the many deadlocked threads "caused" the deadlock.

PARTICIPATION ACTIVITY 5.14.1: Section review questions.

A wait-for graph scheme is not applicable to a resource allocation system with multiple instances of each resource type.

- True
- False

Section glossary

wait-for graph: In deadlock detection, a variant of the resource-allocation graph with resource nodes removed; indicates a deadlock if the graph contains a cycle

thread dump: In Java, a snapshot of the state of all threads in an application; a useful debugging tool for deadlocks.

5.15 A system model for deadlocks

Resource allocation graphs

A system consists of a set of hardware and software resources used by processes. Resources can be divided into reusable and consumable. A reusable resource may be requested, acquired, and later released by a process. Ex: processors, devices, memory blocks, and files. A consumable resource is created by a process and consumed by another process. Ex: messages, interrupts, or signals. This chapter deals with only reusable resources.

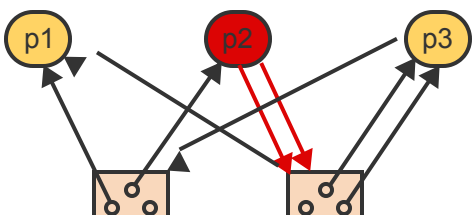
A ***resource allocation graph*** shows the current allocation of resources to processes and the current requests by processes for new resources.

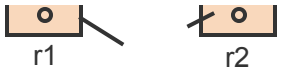
- Processes are represented by circles.
- Resources are represented by rectangles. If a resource contains multiple units then each unit is represented by a small circle.
- Resource allocations are represented by edges directed from a resource to a process.
- Resource requests are represented by edges directed from a process to a resource.

A process p is ***blocked*** on a resource r if one or more request edges directed from p to r exist and r does not contain sufficient free units to satisfy all requests.

The assumption in deadlock analysis and management is that the underlying code is correct. With erroneous code, a process could block itself, for example by acquiring a resource and then requesting the same resource again without releasing the first allocation. Similarly, a process could acquire k units of an n-unit resource and then request m additional units where m > n - k, thus blocking itself forever.

PARTICIPATION ACTIVITY 5.15.1: A resource allocation graph.





Animation content:

Static figure: Three processes, labeled as p1, p2, and p3 with two resources, labeled as r1 and r2. Each resource contains four identical units. Various arrows point to or from the processes and the resources.

Step 1: Three processes are sharing 2 resources, each containing 4 identical units. No arrows. All processes are blue.

Step 2: p1 is holding 1 unit of r1 and 1 unit of r2. p2 is holding 1 unit of r1. p3 is holding 2 units of r2. Allocation arrows point from the resource units to the corresponding processes.

Step 3: p1 has no outstanding requests and thus is not blocked. P1 turns yellow.

Step 4: p2 has outstanding requests for 2 units of r2. Since only 1 unit is available, p2 is blocked. Two request arrows from p2 to r2. P2 turns red.

Step 5: p3 has an outstanding request for 1 unit of r1. Since units of r1 are still available, p3 is not blocked. P3 turns yellow.

Animation captions:

1. Three processes are sharing 2 resources, each containing 4 identical units.
2. p1 is holding 1 unit of r1 and 1 unit of r2. p2 is holding 1 unit of r1. p3 is holding 2 units of r2.
3. p1 has no outstanding requests and thus is not blocked.
4. p2 has outstanding requests for 2 units of r2. Since only 1 unit is available, p2 is blocked.
5. p3 has an outstanding request for 1 unit of r1. Since units of r1 are still available, p3 is not blocked.

PARTICIPATION ACTIVITY | 5.15.2: Valid allocation and request edges.

3 processes are sharing 3 resources:
Resource r1 has 1 unit.
Resource r2 has 2 units.
Resource r3 has 3 units.

Process p1 is holding 1 unit of r1.
Process p3 is holding 2 units of r3.

1) p2 could be currently holding _____.

- 1 unit of r1
- 1 unit of r2
- 2 units of r3

2) p2 could have outstanding request edges for _____.

- 3 units of r1
- 3 units of r2
- 3 units of r3

3) p3 could have outstanding request edges for _____.

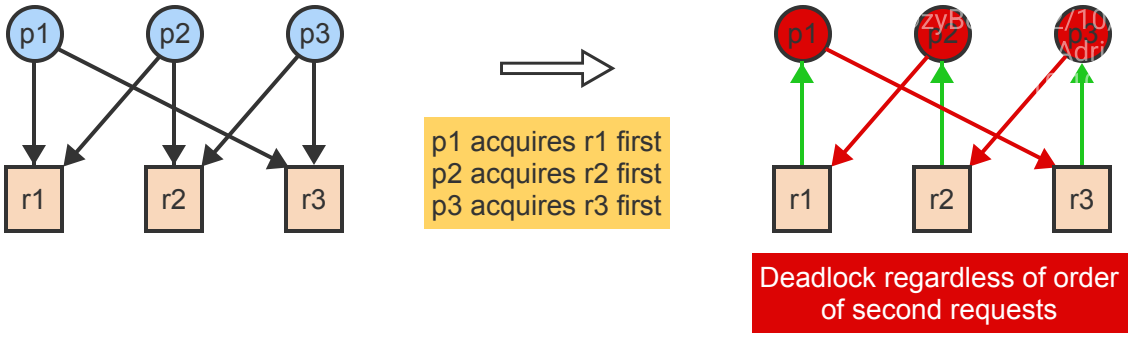
- 1 unit r3
- 2 units r3
- 3 units r3

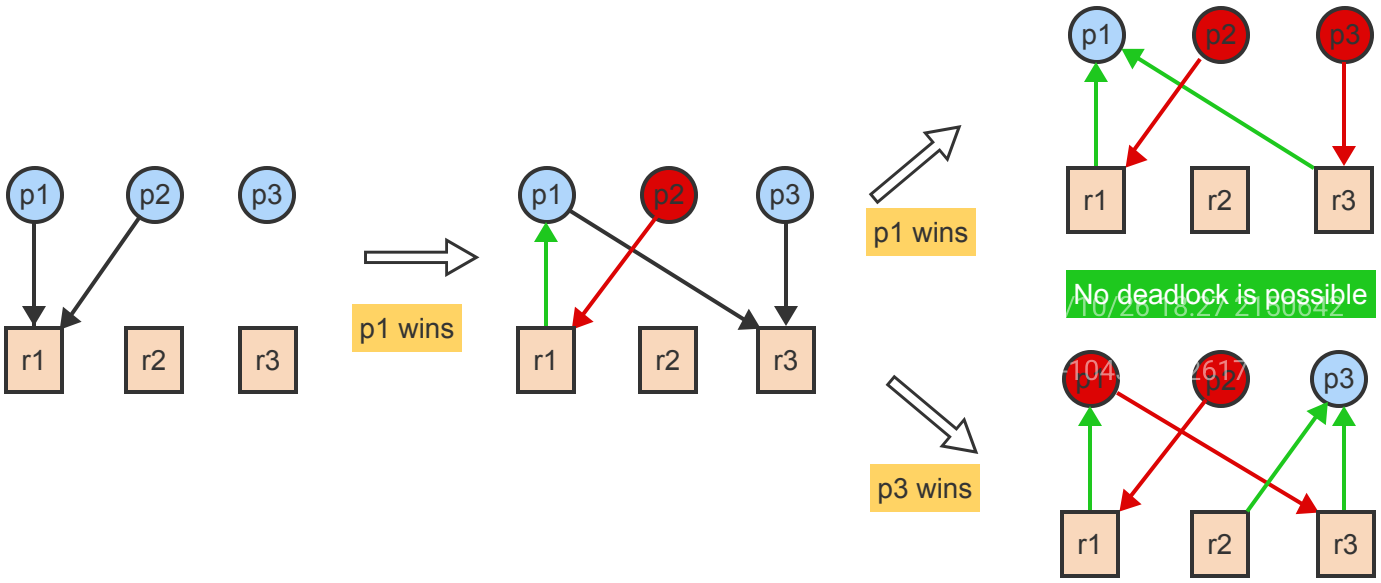
Modeling Deadlocks

A resource r contains one or more identical units, each of which may be requested and used by a process on a non-shared basis.

A deadlock involves at least 2 processes and 2 resources (or resource units), where each process holds one resource and is blocked indefinitely on another resource held by another process.

PARTICIPATION ACTIVITY 5.15.3: Possible deadlock with 3 processes and 3 single-unit resources.





Animation content:

Static figure: A deadlock scenario involving three processes (p1, p2, p3) and three resources (r1, r2, r3). Directed lines connect processes to the required resources, indicating which process is requesting which resource. The same set of 3 processes and 3 resources is replicated several times on the page to illustrate different scenarios.

Step 1: Three processes are sharing 3 resources. p1 needs r1 and r3, p2 needs r1 and r2, and p3 needs r2 and r3. Corresponding request arrows from processes to resources appear.

Step 2: One possibility is that p1 requests and acquires r1 first, p2 requests and acquires r2 first, and p3 requests and acquires r3 first. Corresponding request arrows reverse direction to become allocation arrows.

Step 3: Then if all 3 processes request a second resource, all become blocked in a deadlock situation. All 3 processes turn red.

Step 4: If p2 requests the resources in reverse order then processes p1 and p2 compete for the same resource (r1) during the first request. New scenario. Request arrows from p1 and p2 to r1.

Step 5: If p1 wins then p2 is blocked and p1 and p3 compete for r3. (An analogous situation would occur if p2 won the competition for r1.) Request arrow p1 to r1 is reversed to become allocation arrow. P2 turns red. Request arrows from p1 and p3 to r3 appear.

Step 6: If p1 wins again then p2 and p3 are blocked while p1 continues with both r1 and r3. When p1 terminates, p2 or p3 can continue. No deadlock is possible. Request arrow p1 to r3 is reversed to become allocation arrow. P3 turns red.

Step 7: If p3 wins instead, then p1 and p2 are blocked while p3 continues with both r2 and r3. When p3 terminates, p1 or p2 can continue. Again, no deadlock is possible. Allocation arrows from r2 and r3 to p3 mean that p3 is not blocked. P1 and p2 are red.

Animation captions:

1. Three processes are sharing 3 resources. p1 needs r1 and r3, p2 needs r1 and r2, and p3 needs r2 and r3.
2. One possibility is that p1 requests and acquires r1 first, p2 requests and acquires r2 first, and p3 requests and acquires r3 first.
3. Then if all 3 processes request a second resource, all become blocked in a deadlock situation.
4. If p2 requests the resources in reverse order then processes p1 and p2 compete for the same resource (r1) during the first request.
5. If p1 wins then p2 is blocked and p1 and p3 compete for r3. (An analogous situation would occur if p2 won the competition for r1.)
6. If p1 wins again then p2 and p3 are blocked while p1 continues with both r1 and r3. When p1 terminates, p2 or p3 can continue. No deadlock is possible.
7. If p3 wins instead, then p1 and p2 are blocked while p3 continues with both r2 and r3. When p3 terminates, p1 or p2 can continue. Again, no deadlock is possible.

PARTICIPATION ACTIVITY 5.15.4: Number of processes and resources needed for a deadlock.

Deadlock is possible when

- the number of processes is greater than 1, regardless of the number of resources.
- the number of resources is greater than 1, regardless of the number of processes.
- the number of processes and the number of resources are both greater than 1.

PARTICIPATION ACTIVITY 5.15.5: Order of requests leading to deadlock.

Three processes are sharing 3 resources:
p1 requests: r1 and then r2
p2 requests: r2 and then r3

p3 requests: r3 and then r1
Assuming all processes run concurrently and request resources at the same time, deadlock will occur.

Deadlock is also possible if

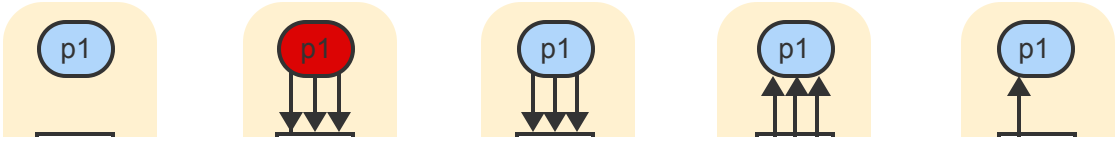
- p1 requests the resources in reverse order.
- p1 and p2 request the resources in reverse order.
- all 3 processes request the resources in reverse order.

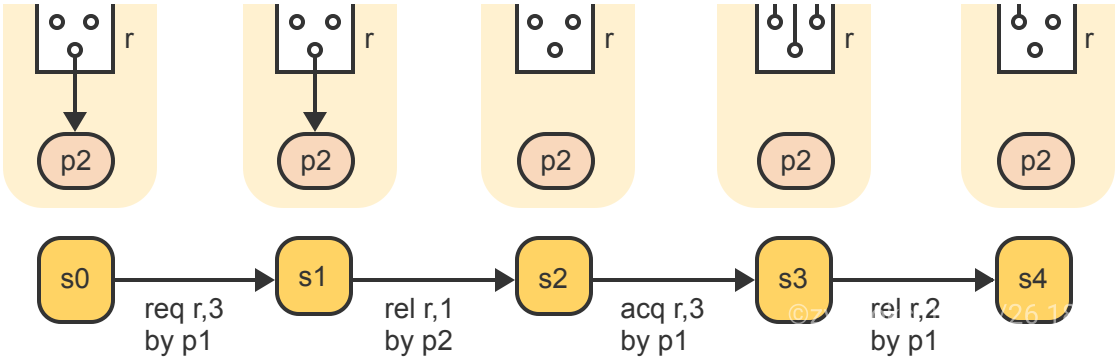
State transitions

A resource allocation graph represents the current state of the system. Three operations move the system into a new state, represented by a different resource allocation graph:

- A ***resource request*** (req r, m) by a process p for m units of a resource r creates m new edges directed from p to r. A resource request may be issued only when:
  1. Process p has no unsatisfiable requests and thus is not blocked. A blocked process is not running and cannot issue additional requests.
  2. The number m of request edges plus the number k of allocation edges between a process and a resource must not exceed the total number of units of the resource. Exceeding m + k implies erroneous code, where a process failed to perform a release prior to a new request.
- A ***resource acquisition*** (acq r, m) by a process p of m units of a resource r reverses the direction of the corresponding request edges to point from the units of r to p. A resource acquisition is executed by the system and is allowed only when r currently contains at least m free units. No partial allocation of units is permitted.
- A ***resource release*** (rel r, m) operation by a process p of m units of a resource r deletes m allocation edges between p and r. A resource release may be issued only when:
  1. Process p has no unsatisfiable requests and thus is not blocked.
  2. The number m of units to be released must not exceed the total number of units p is currently holding.

PARTICIPATION ACTIVITY 5.15.6: A sequence of possible state transitions.





Animation content:

Static figure: A series of state transitions s0 through s4 triggered by request, acquire, and release operations. Each state shows a resource allocation graph with two processes, p1 and p2, and a single resource r containing 3 units.

Step 1: In state s0, process p2 is holding 1 unit of resource r. Arrow from one unit of r to p2.

Step 2: Process p1 requests 3 units of resource r.

Step 3: The system enters a new state s1, where p1 is blocked because only 2 units of r are available. 3 arrows point from p1 to r. r is red, indicating a blocked state.

Step 4: p2 releases one unit of r, taking the system into a new state s2, where p1 is no longer blocked because all 3 units are available. Arrow from r to p2 is removed. P1 turns blue (not blocked).

Step 5: p1 can now acquire all 3 units of r and the system enter a new state s3. Alternately, p2 could request 1, 2, or 3 units of r, each of which would take the system into a different state. The 3 arrows from p1 to r reverse direction (become allocation arrows)

Step 6: In state s3, p2 could again request 1, 2, or 3 units of r, or p1 could release 1, 2, or 3 units of r, each leading to a new state. If p1 releases 2 of the 3 units then both processes remain unblocked in state s4. 2 arrows from r to p1 are removed.

Animation captions:

1. In state s0, process p2 is holding 1 unit of resource r.
2. Process p1 requests 3 units of resource r.
3. The system enters a new state s1, where p1 is blocked because only 2 units of r are available.

4. p2 releases one unit of r, taking the system into a new state s2, where p1 is no longer blocked because all 3 units are available.
5. p1 can now acquire all 3 units of r and the system enters a new state s3. Alternately, p2 could request 1, 2, or 3 units of r, each of which would take the system into a different state.
6. In state s3, p2 could again request 1, 2, or 3 units of r, or p1 could release 1, 2, or 3 units of r, each leading into a new state. If p1 releases 2 of the 3 units then both processes remain unblocked in state s4.

PARTICIPATION ACTIVITY 5.15.7: Validity of operations depends on the state.

1) In state s0 of the above graph, _____.

- p1 could release r,1
- p2 could request r,3
- p2 could request r,1

2) In state s1, _____.

- p1 could release r,3
- p1 could acquire r,2
- p2 could request r,2

3) In state s2, _____.

- p1 could release r,3
- p1 could acquire r,1
- p2 could request r,3

4) In state s3, _____.

- p1 could request r,1
- p2 could request r,3
- p2 could acquire r,1

Deadlock states and safe states

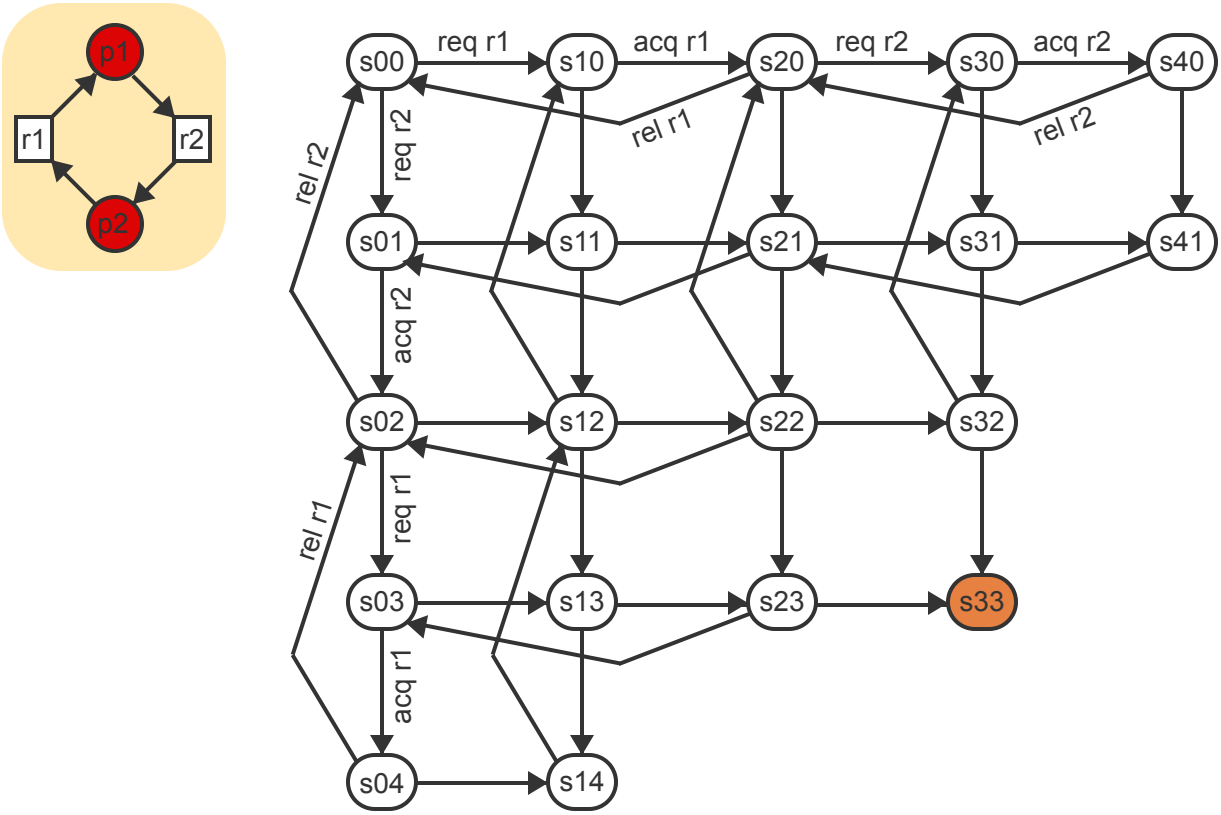
A process is ***deadlocked*** in a state s if the process is blocked in s and if no matter what state transitions occur in the future, the process remains blocked.

A state s is called a ***deadlock state*** if s contains two or more deadlocked processes.

A state s is a ***safe state*** if no sequence of state transitions exists that would lead from s to a deadlock state.

When the sequence of all requests and releases by each process is known, a complete state transition graph can be constructed and analyzed for the presence of deadlock states and safe states.

PARTICIPATION ACTIVITY 5.15.8: A deadlock state in a state transition graph.



Process p1 and p2 share single-unit resources r1 and r2. p1 always requests r1 before r2 and releases r2 before r1. p2 always requests r2 before r1 and releases r1 before r2.

Animation content:

Static figure: First part of diagram shows a resource allocation graph consisting of processes p1 and p2 and resources r1 and r2. The second part shows a state transition diagram for the two processes as they compete for the two resources. Each process can either request (req), acquire (acq), or release (rel) one of the resources. Each operation transitions into a new state. These states are represented by nodes, labeled s00 through s40 in the horizontal direction and s00 through s04 in the vertical direction, with s33 being highlighted as a deadlock state. Arrows labeled with the operations indicate the transitions between states.

Step 1: Initially, in state s00, no resources are allocated to either p1 or p2. There are no arrows in the allocation graph. The allocation corresponds to state S00 in the state transition graph. S00 appears.

Step 2: When p2 is not using any resources, p1 can repeatedly request, acquire, and release both r1 and r2 in the specified order. Req r1 leads from s00 to state s10. Ack r1 leads to the next state s20. From s20 two directions are possible: Rel r1 leads back to s00, while req r2 leads to s30. Ack r2 leads from s30 to s40. Rel r2 leads from s40 back to s20.

Step 3: Similarly, when p1 is not using any resources, p2 can repeatedly request, acquire, and release both r2 and r1 in the specified order. An analogous series of state transitions goes vertically from s00 down to s04.

Step 4: When p1 and p2 execute concurrently, other states can be reached, each representing a different snapshot of requests and allocations by the two processes. The remainder of the rectangular grid of state transitions is filled in.

Step 5: In state s04, p2 is holding both r2 and r1. p1 has not taken any action and thus has no edges to either resource. In the resource allocation graph, both resources point to p2. In the state transition graph, s04 is highlighted.

Step 6: If p1 now requests r1, p1 becomes blocked in state s14 because r1 is already held by p2 and cannot be acquired by p1. No horizontal transition out of s14 exists. In the resource allocation graph, p1 points to r1 and turns red. In the state transition graph, s14 is highlighted.

Step 7: When p2 releases r1, both processes are again unblocked in state s12. In the resource allocation graph, arrow from r1 to p2 disappears and p1 turns blue. In the state transition graph, s12 is highlighted.

Step 8: p1 can now acquire r1 and request r2, becoming blocked again in s32. R1 points to p1 and p1 points to r2. P1 turns red. In the state transition graph, s32 is highlighted.

Step 9: If p2 now requests r1, the cycle closes and both processes become blocked in s33. State s33 is a deadlock state. P2 points to r1. S33 is marked red.

Animation captions:

1. Initially, in state s00, no resources are allocated to either p1 or p2.
2. When p2 is not using any resources, p1 can repeatedly request, acquire, and release both r1 and r2 in the specified order.
3. Similarly, when p1 is not using any resources, p2 can repeatedly request, acquire, and release both r2 and r1 in the specified order.
4. When p1 and p2 execute concurrently, other states can be reached, each representing a different snapshot of requests and allocations by the two processes.
5. In state s04, p2 is holding both r2 and r1. p1 has not taken any action and thus has no edges to either resource.
6. If p1 now requests r1, p1 becomes blocked in state s14 because r1 is already held by p2 and cannot be acquired by p1. No horizontal transition out of s14 exists.

7. When p2 releases r1, both processes are again unblocked in state s12.
8. p1 can now acquire r1 and request r2, becoming blocked again in s32.
9. If p2 now requests r1, the cycle closes and both processes become blocked in s33. State s33 is a deadlock state.

PARTICIPATION ACTIVITY 5.15.9: Interpreting the state transition graph.

1) Operations by p1 correspond to _____ in the transition state graph.
   - horizontal edges
   - downward-pointing edges
   - upward-ponting edges

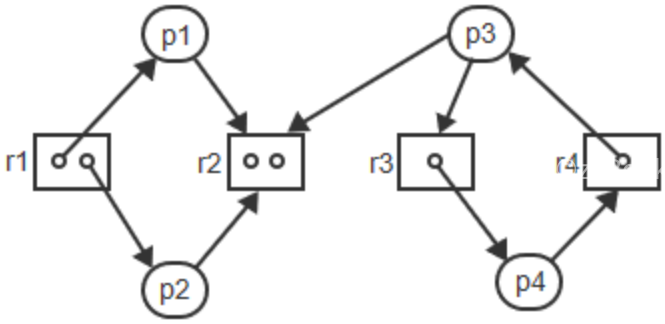
2) In which states is p1 blocked?
   - s41
   - s32, s33, and s14
   - only s33

3) Which states are safe states?
   - s00
   - all except s32 and s23
   - none

4) In state s22 _____.
   - p1 is holding r1 and p2 is holding r2
   - p1 is holding r2 and p2 is holding r1
   - p1 is requesting r1 and p2 is requesting r2

5) In state s31 _____.
   - p1 is requesting r1 and p2 is holding r2 and requesting r1
   - p1 is holding r1 and requesting r2, and p2 is requesting r2
   - p1 is holding r1 and requesting r2, and p2 is holding r2

EXERCISE 5.15.1: Understanding a resource allocation graph.



(a) Which processes are blocked?

(b) Which processes are deadlocked?

EXERCISE 5.15.2: Modified state transition diagram.

The state transition diagram in the Participation Activity titled "A deadlock state in a state transition graph" represents the behavior of two processes sharing two resources.

(a) Modify the diagram such that each process acquires the second resource without releasing the first and then releases both at the same time.

(b) Modify the diagram such that process p1 only needs resource r2.

(c) Does either modification eliminate the possibility of deadlock?

EXERCISE 5.15.3: Modeling process interactions.

p1:

```
while(1) {
   request r1
   request r2
   release r1
   release r2
   ... }
```

p2:

```
while(1) {
   request r2
   request r1
   release r2
   release r1
   ... }
```

(a) Draw a state transition diagram to model the above interactions.

(b) Draw the resource graphs corresponding to states s10, s11, s12, s13, s14, s15, s23, s33.

EXERCISE | 5.15.4: Reversing order of operations.

(a) Modify the previous problem such that p2 requests and releases the resources in the same order as p1. Draw the modified diagram.

(b) In which states are the processes blocked or deadlocked?

5.16 Classic synchronization problems

The readers-writers problem

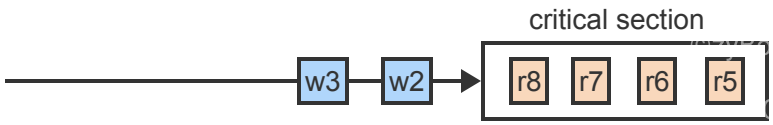
The readers-writers problem is an extension of the critical section problem where two types of processes, readers and writers, compete for access to a common resource. Readers are allowed to enter the critical section (CS) concurrently but only one writer is allowed to enter at a time.

The main challenge is to guarantee maximum concurrency of readers while preventing the starvation of either type of process. Specifically, two rules must be enforced:

1. A reader is permitted to join other readers currently in the CS only when no writer is waiting. When the last readers exits the CS, the writer is allowed to enter.
2. All readers that have arrived while a writer is in the CS must be allowed to enter before the next writer.

Rule 1 guarantees that writers cannot starve. Rule 2 guarantees that readers cannot starve. Jointly the two rules guarantee maximum concurrency of readers.

PARTICIPATION ACTIVITY | 5.16.1: Order of readers and writers entering the critical section.



Animation content:

Static figure: A diagram of the readers/writer problem. Readers (r8, r7, r6, r5) are in the critical section (CS) while writers (w3, w2) wich to enter.

Step 1: Three readers arrive at the CS. Since the CS is empty, all three are allowed to enter.

Step 2: A fourth reader arrives. Since no writer is waiting, r4 is allowed to join the other readers.
A fourth reader, r4, joins the previously present readers inside the CS. No writers are waiting.

Step 3: When a writer arrives, no more readers are allowed to enter.
A writer, w1, has arrived and is queued outside the CS, with additional readers r5 and r6 behind it. Readers already in the CS remain there, but the entry of new readers is now restricted.

Step 4: When the last reader leaves, w1 is allowed to enter.
All readers have left the CS, and writer w1 is now inside the CS.

Step 5: While w1 is in CS, other readers and writers may arrive.
Writer w1 inside the CS with new readers r5 through r8 and writers w2 and w3 queued outside.

Step 6: When w1 leaves, all readers, including those arriving after w2 and w3, are allowed to enter.
Writer w1 has left the CS, and readers r5 through r8 now occupy the CS. Writers w2 and w3 remain queued outside.

Animation captions:

1. Three readers arrive at the CS. Since the CS is empty, all three are allowed to enter.
2. A fourth reader arrives. Since no writer is waiting, r4 is allowed to join the other readers.
3. When a writer arrives, no more readers are allowed to enter.
4. When the last reader leaves, w1 is allowed to enter.
5. While w1 is in CS, other readers and writers may arrive.
6. When w1 leaves, all readers, including those arriving after w2 and w3, are allowed to enter.

PARTICIPATION ACTIVITY 5.16.2: Order of reader and writer processing.

1) While w1 is in the CS, the following processes arrive:
r1, r2, w2, w3, r3.
The processes will enter the CS in the order:

- w1, r1, r2, w2, w3, r3
- w1, r1, r2, r3, w2, w3
- w1, r1, r2, w2, r3, w3

2) While r1 is in the CS, the following processes arrive:
r2, w1, w2, r3, r4.
The processes will enter the CS in the order:

- r1, w1, r2, w2, r3, r4
- r1, r2, w1, w2, r3, r4
- r1, r2, w1, r3, r4, w2

PARTICIPATION ACTIVITY 5.16.3: A modified rule for readers.

Changing the second rule of the requirements to "All readers that arrive between any two writers are allowed to proceed concurrently" could:

- lead to starvation of readers
- lead to starvation of writers
- decrease maximum concurrency of readers

A monitor solution to the readers-writers problem

The monitor provides 4 functions:

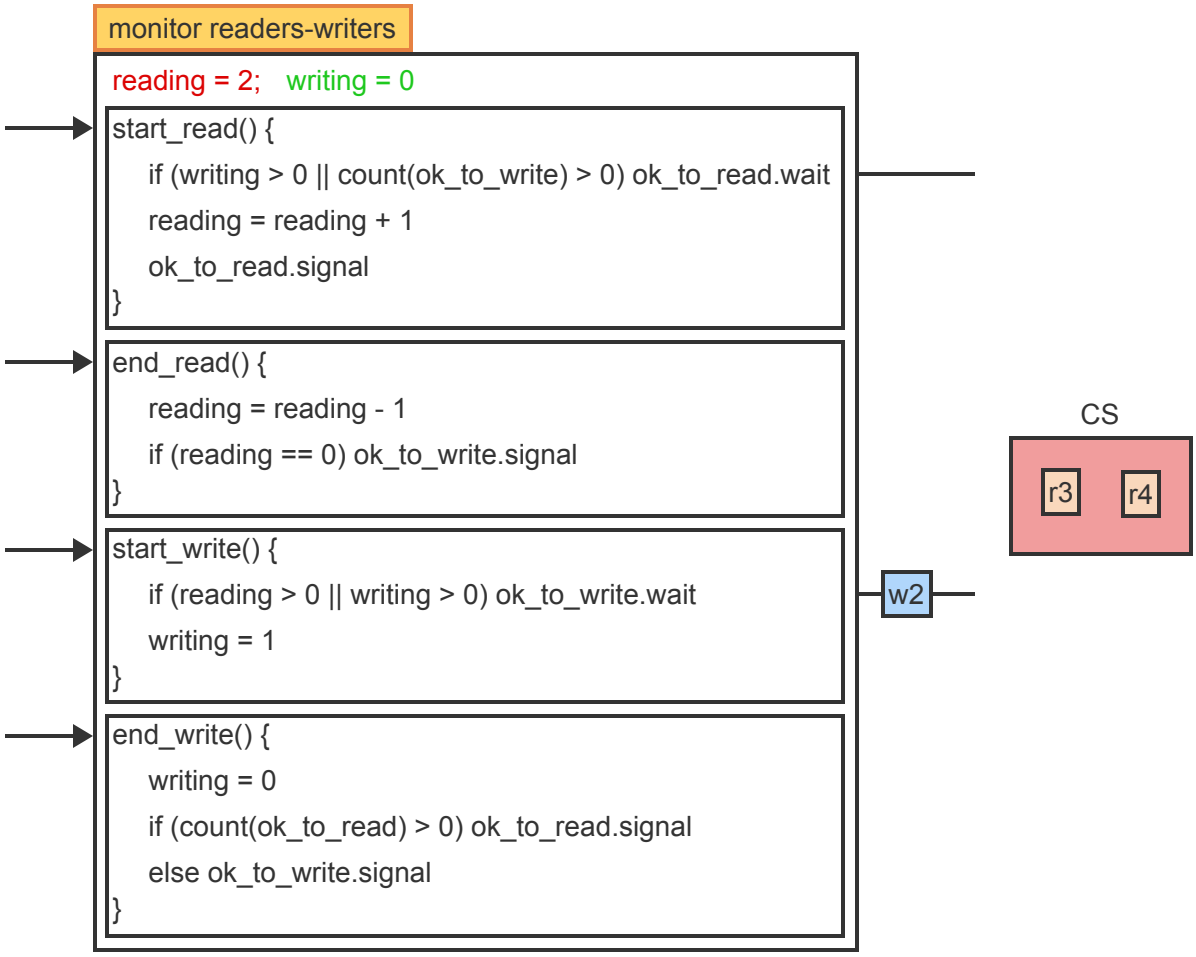
- start_read is called by a reader to get a permission to read
- end_read is called by a reader when finished reading
- start_write is called by a writer to get a permission to write
- end_write is called by a writer when finished writing

Two counters, reading and writing, are used to keep track of the number of readers and the number of writers currently in CS, respectively.

Two condition variables, ok_to_read and ok_to_write, are used to block readers and writers, respectively.

To avoid maintaining separate counters of processes blocked on a condition c, a primitive count(c) is provided, which returns the number of processes blocked on c. Ex: count(ok_to_read) returns the number of processes blocked on ok_to_read.

PARTICIPATION ACTIVITY 5.16.4: Readers-writers problem using a monitor.



Animation content:

Static figure: A monitor solution to the readers-writers problem in a concurrent system. Pseudocode within the monitor includes two global counters, reading=2 and writing= 0, and four functions: start_read(), end_read(), start_write(), and end_write(). The Critical Section (CS) is represented as a

rectangle next to the monitor.

Begin pseudocode of monitor.

```
monitor readers-writers
reading = 2; writing = 0

start_read() {
	if (writing > 0 || count(ok_to_write) > 0) ok_to_read.wait
	reading = reading + 1
	ok_to_read.signal
}

end_read() {
	reading = reading - 1
	if (reading == 0) ok_to_write.signal
}

start_write() {
	if (reading > 0 || writing > 0) ok_to_write.wait
	writing = 1
}

end_write() {
	writing = 0
	if (count(ok_to_read) > 0) ok_to_read.signal
	else ok_to_write.signal
}
```

End pseudocode.

Step 1: A reader r1 calls start_read(). Since no writer or reader is in the CS, r1 increments the reading counter and enters the CS. The ok_to_read.signal has no effect since the queue is empty. R1, represented as a small box moves to CS box. Reading=1.

Step 2: A second reader, r2, arrives. The conditions are the same and thus r2 joins r1 in the CS. R2 moves to CS box. Reading=2.

Step 3: A writer w1 arrives and calls start_write. Since reading > 0, w1 blocks on ok_to_write. W1 waits outside on queue ok_to_write.

Step 4: The next reader, r3, finds the queue ok_to_write not empty and so r3 blocks on ok_to_read. R3 steps outside and enters queue ok_to_read.

Step 5: r1 leaves the CS and calls end_read. The reading counter is decremented but r1 is not the last process to leave and thus no signal is issued. Reading=1

Step 6: When r2 leaves the CS, reading becomes 0. Since r2 is the last process to leave the CS, r2 signals to w1 to resume. Since the signal is the last operation, r2 leaves the monitor without blocking.

Step 7: w1 re-enters, sets writing to 1, and enters CS.

Step 8: While w1 is in the CS, a reader r4 and a writer w2 arrive. Since writing = 1, both processes get blocked on the respective queues.

Step 9: When w1 leaves the CS, a signal enables r3 to re-enter.

Step 10: Upon re-entering, r3 signals to the next reader on the queue, r4, which, in turn would signal to the next reader on the queue until all readers accumulated thus far had entered the CS.

Animation captions:

1. A reader r1 calls start_read(). Since no writer or reader is in the CS, r1 increments the reading counter and enters the CS. The ok_to_read.signal has no effect since the queue is empty.
2. A second reader, r2, arrives. The conditions are the same and thus r2 joins r1 in the CS.
3. A writer w1 arrives and calls start_write. Since reading > 0, w1 blocks on ok_to_write.
4. The next reader, r3, finds the queue ok_to_write not empty and so r3 blocks on ok_to_read.
5. r1 leaves the CS and calls end_read. The reading counter is decremented but r1 is not the last process to leave and thus no signal is issued.
6. When r2 leaves the CS, reading becomes 0. Since r2 is the last process to leave the CS, r2 signals to w1 to resume. Since the signal is the last operation, r2 leaves the monitor without blocking.
7. w1 re-enters, sets writing to 1, and enters CS.
8. While w1 is in the CS, a reader r4 and a writer w2 arrive. Since writing = 1, both processes get blocked on the respective queues.
9. When w1 leaves the CS, a signal enables r3 to re-enter.
10. Upon re-entering, r3 signals to the next reader on the queue, r4, which, in turn would signal to the next reader on the queue until all readers accumulated thus far had entered the CS.

PARTICIPATION ACTIVITY 5.16.5: Type of variables used.

1) ok_to_read and ok_to_write are

- regular variables
- condition variables
- semaphores

2) reading is an integer variable where

- 0 ≤ reading
- 1 ≤ reading
- 0 ≤ reading ≤ 1

3) writing is an integer variable where

- 0 ≤ writing
- 1 ≤ writing
- 0 ≤ writing ≤ 1

PARTICIPATION ACTIVITY 5.16.6: Cases of blocking and signaling.

1) If the CS is empty, then an arriving writer blocks subsequent readers from entering the CS by setting writing = 1.

- True
- False

2) If readers are currently in the CS, then an arriving writer blocks subsequent readers from entering the CS by setting writing = 1.

- True
- False

3) An exiting writer signals the next writer if ok_to_read queue is empty.

- True
- False

4) An exiting writer signals all readers currently on the ok_to_read queue.

- True
- False

5) Each reader re-entering from the ok_to_read queue signals the next reader on the same queue before entering the CS.

- True
- False

6) Each exiting reader signals the next writer.

- True
- False

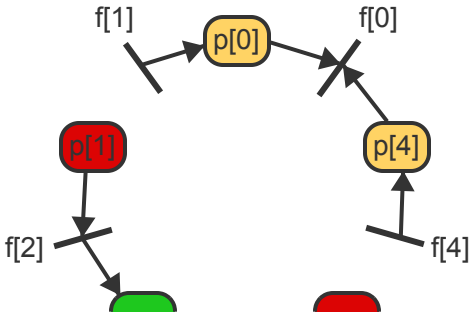
The dining-philosophers problem

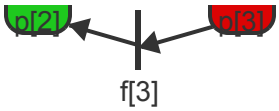
The dining-philosophers problem is representative of situations where multiple processes compete for shared resources but each process needs more than one resource at a time.

Five "philosophers", each representing a concurrent process, are seated around a table. Five "forks", each representing a resource, are placed on the table such that each two neighboring philosophers share one fork. Each philosopher alternates asynchronously between a phase of "thinking", which represents execution not requiring any shared resources, and "eating", which requires the prior acquisition of the two forks adjacent to the philosopher and shared with the two respective neighbors.

The main challenge is to prevent deadlock while guaranteeing that any two nonadjacent philosophers can always eat concurrently.

PARTICIPATION ACTIVITY 5.16.7: Requirements of the dining-philosophers problem.





Animation content:

Static figure: A circle of five philosophers labeled p[0] to p[4] and five forks labeled f[0] to f[4]. The philosophers are evenly spaced around the circle, with a fork placed between each pair. Each philosopher has access to a left fork and a right fork, with the convention that the right fork of p[i] is the left fork of p[(i+1) mod 5]. Initially, all philosophers are in the thinking state.

Step 1: Five philosophers, p[i], sit in a circle. Each p[i] has a left fork, f[i], and a right fork, f[i+1 mod 5]. Initially, all p[i]'s are thinking, which requires no resources. All philosophers are white.

Step 2: In order to eat, a philosopher needs to request both the left and the right fork. Ex: p[2] needs to request f[2] and f[3] to eat. P[2] becomes yellow and arrows point from P[2] to the neighboring forks.

Step 3: If both forks are available, p[i] picks up the forks and starts eating. P[2] becomes green and arrows now point from the forks to p[2].

Step 4: When finished eating, p[i] puts down both forks and enters again the phase of thinking. All philosophers are white again and there are no arrows.

Step 5: To prevent deadlock, a situation where each p[i] picks up one fork and is requesting the second must be avoided, since all p[i]s would be blocked forever. Each philosopher is holding one fork and has an arrow pointing towards the other fork they need, which is currently unavailable because the philosopher next to it is also requesting the same fork. All philosophers are red (blocked).

Step 6: When one philosopher is eating, the two immediate neighbors are blocked because each needs one of the shared forks. Ex: p[2] is holding f[2] and f[3], which blocks p[1] and p[3].
p[2] is green (eating) and arrows point from the neighboring forks f[2] and f[3] toward p[2]. The neighbors p[1] and p[3] are red (blocked), while pointing to the forks f[2] and f[3], unable to proceed to the eating state because they require the unavailable forks that p[2] is currently using.

Step 7: But the solution must guarantee that either p[0] or p[4] can eat concurrently with p[2], depending on which philosopher grabs the shared fork f[0] first. P[0] and p[4] are yellow, both pointing to f[0].

Animation captions:

1. Five philosophers, p[i], sit in a circle. Each p[i] has a left fork, f[i], and a right fork, f[i+1 mod 5]. Initially, all p[i]'s are thinking, which requires no resources.
2. In order to eat, a philosopher needs to request both the left and the right fork. Ex: p[2] needs to request f[2] and f[3] to eat.
3. If both forks are available, p[i] picks up the forks and starts eating.
4. When finished eating, p[i] puts down both forks and enters again the phase of thinking.
5. To prevent deadlock, a situation where each p[i] picks up one fork and is requesting the second must be avoided, since all p[i]s would be blocked forever.
6. When one philosopher is eating, the two immediate neighbors are blocked because each needs one of the shared forks. Ex: p[2] is holding f[2] and f[3], which blocks p[1] and p[3].
7. But the solution must guarantee that either p[0] or p[4] can eat concurrently with p[2], depending on which philosopher grabs the shared fork f[0] first.

PARTICIPATION ACTIVITY 5.16.8: Requirements of the dining-philosophers problem.

1) When no philosopher is eating then

- only 1 can start eating
- at most 2 can start eating
- at most 3 can start eating

2) When p[4] is eating then _____ can eat concurrently.

- only p[1]
- p[0] or p[1]
- p[1] or p[2]

3) Deadlock can occur _____.

- only when the number of philosophers is odd
- only when the number of philosophers is exactly 5
- whenever the number of philosophers is greater than 1

Approaches to preventing deadlock

The behavior of each philosopher p[i] can be represented as a loop that alternates between the phases of thinking and eating. Prior to eating, p[i] requests the two adjacent forks and returns the forks when finished

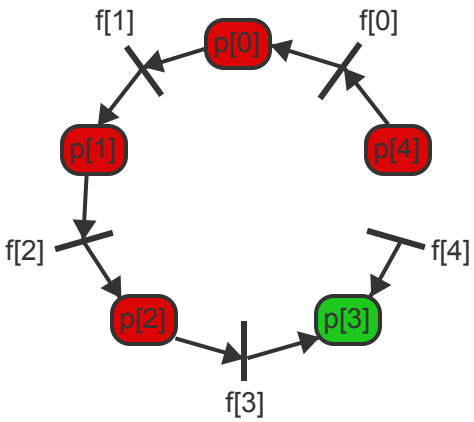
eating. The forks can be represented as 5 semaphores, f[0] through f[4], all initialized to 1. P(f[i]) then corresponds to picking up fork f[i] and V(f[i]) corresponds to putting down fork f[i].

```
p(i) {
   while (1) {
          think
          P(f[i])
          P(f[i+1 mod 5])
          eat
          V(f[i])
          V(f[i+1 mod 5])
   }
}
```


This code leads to deadlock since all philosophers can pick up the left fork f[i] concurrently and then block indefinitely on picking up the right fork. Several approaches exist to avoid the problem:

- Approach 1: Request both forks at the same time in a critical section.
- Approach 2: One philosopher picks up the forks in the opposite order from all other philosophers.

PARTICIPATION ACTIVITY 5.16.9: Preventing deadlock in the dining-philosophers problem.



Approach 1

```
p(i) {
  while (1) {
    think
    P(mutex)
    P(f[i])
    P(f[i+1 mod 5])
    V(mutex)
    eat
    V(f[i])
    V(f[i+1 mod 5])
  }
}
```

Approach 2

```
p(i) {
  while (1) {
    think
    P(f[min(i, i+1 mod 5)])
    P(f[max(i, i+1 mod 5)])
    eat
    V(f[i])
    V(f[i+1 mod 5])
  }
}
```

Animation content:

Static figure: A circle of five philosophers, labeled p[0] through p[4], sitting around a table with five forks, labeled f[0] through f[4], placed between each pair of them. Two code blocks, Approach 1 and Approach 2 show pseudocode for handling the philosophers' actions to avoid deadlock. In Approach 1, the code is as follows:

```
p(i) {
   while (1) {
           think
           P(mutex)
           P(f[i])
           P(f[(i+1 mod 5)])
           V(mutex)
           eat
           V(f[i])
           V(f[(i+1 mod 5)])
   }
}
```


In Approach 2, the code is updated to:

```
p(i) {
   while (1) {
           think
           P(f[min(i, (i+1 mod 5))])
           P(f[max(i, (i+1 mod 5))])
           eat
           V(f[i])
           V(f[(i+1 mod 5)])
   }
}
```

Step 1: One approach to preventing all philosophers from picking up the left fork first and becoming deadlocked is to force both forks to be picked up at the same time in a critical section. P(mutex) and V(mutex) surround the two instructions P(f[i]) and P(f[(i+1 mod 5)]).

Step 2: When one philosopher, p[2], has picked up the left fork, mutex is 1 and all other philosophers get blocked on P(mutex). p[2] is able to request and get the right fork and thus deadlock is prevented. P[2] is yellow (requesting state), holding f[2] and requesting f[3], while all other philosophers are red (blocked).

Step 3: But concurrency may be violated as shown in the following steps:
When p[0] and p[2] are eating, p[3] is blocked on the left fork and p[1] and p[4] are blocked on P(mutex). P[0] and p[2] are green (eating). All others are red (blocked).

Step 4: When p[0] puts down the forks, p[4] remains blocked on mutex. Thus the solution violates the concurrency requirements: p[4] should be allowed to eat concurrently with p[2]. P[0] turns white, while p[4] remains red even though both neighboring forks are available.

Step 5: With approach 2, each philosopher picks up the lower-numbered fork first. p[0] through p[3] pick up f[0] through f[3] first, respectively. But p[4] picks up the right fork f[0] before the left fork f[4] because 0<4. All philosophers are yellow. P[4] points to its right fork f[0] while all others point to their left fork.

Step 6: The new order prevents deadlock because the cycle cannot be closed regardless of whether p[0] or p[4] picks up f[0] first. P[1] and p[2] are red. All others are yellow. P[0] and p[4] point to the same shared fork f[0].

Step 7: If p[0] wins and picks up f[0] first then both p[0] and p[4] get blocked and p[3] can continue. P[3] is green, all other are red.

Step 8: If p[4] wins and picks up f[0] first then p[4] or p[3] each hold one fork and compete for f[4]. Regardless of which of the two wins, one can continue and thus deadlock is not possible. P[3] and p[4] are yellow, pointing to the same shared fork f[4]. All others are red.

Step 9: But concurrency can still be violated. When p[3] is eating, all others can become blocked in a chain waiting for each other. Thus neither p[0] nor p[1] can eat concurrently with p[3]. P[3] is green. All others are red.

Animation captions:

1. One approach to preventing all philosophers from picking up the left fork first and becoming deadlocked is to force both forks to be picked up at the same time in a critical section.
2. When one philosopher, p[2], has picked up the left fork, mutex is 1 and all other philosophers get blocked on P(mutex). p[2] is able to request and get the right fork and thus deadlock is prevented.
3. But concurrency may be violated as shown in the following steps: When p[0] and p[2] are eating, p[3] is blocked on the left fork and p[1] and p[4] are blocked on P(mutex).
4. When p[0] puts down the forks, p[4] remains blocked on mutex. Thus the solution violates the concurrency requirements: p[4] should be allowed to eat concurrently with p[2].
5. With approach 2, each philosopher picks up the lower-numbered fork first. p[0] through p[3] pick up f[0] through f[3] first, respectively. But p[4] picks up the right fork f[0] before the left fork f[4] because 0<4.
6. The new order prevents deadlock because the cycle cannot be closed regardless of whether p[0] or p[4] picks up f[0] first.
7. If p[0] wins and picks up f[0] first then both p[0] and p[4] get blocked and p[3] can continue.
8. If p[4] wins and picks up f[0] first then p[4] or p[3] each hold one fork and compete for f[4]. Regardless of which of the two wins, one can continue and thus deadlock is not possible.
9. But concurrency can still be violated. When p[3] is eating, all others can become blocked in a chain waiting for each other. Thus neither p[0] nor p[1] can eat concurrently with p[3].

PARTICIPATION ACTIVITY 5.16.10: States where concurrency is violated.

Each table represents a snapshot of the dining-philosophers problem.

A:

	holding	requesting
p[0]	f[0]	f[1]
p[1]	f[1],f[2]	
p[2]		f[2]
p[3]	f[3]	f[4]
p[4]		f[0]

B:

	holding	requesting
p[0]	f[0]	f[1]
p[1]	f[1]	f[2]
p[2]	f[2],f[3]	
p[3]		f[3]
p[4]		f[0]

C:

	holding	requesting
p[0]	f[0],f[1]	
p[1]		f[1]
p[2]	f[2]	f[3]
p[3]		f[3]
p[4]		f[0]

1) The concurrency requirements is violated in state _____.

- A
- B
- C

2) The only state possible with approach 1 is _____.

- A
- B
- C

PARTICIPATION ACTIVITY 5.16.11: State where concurrency violation depends on order of execution.

	holding	requesting
p[0]		f[0]
p[1]	f[1]	f[2]
p[2]		f[2]
p[3]	f[3]	f[4]

p[4]	f[0],f[4]	

In the above state, p[4] is eating and p[0] and p[3] are already blocked. If _____ continues, the concurrency requirement will be violated.

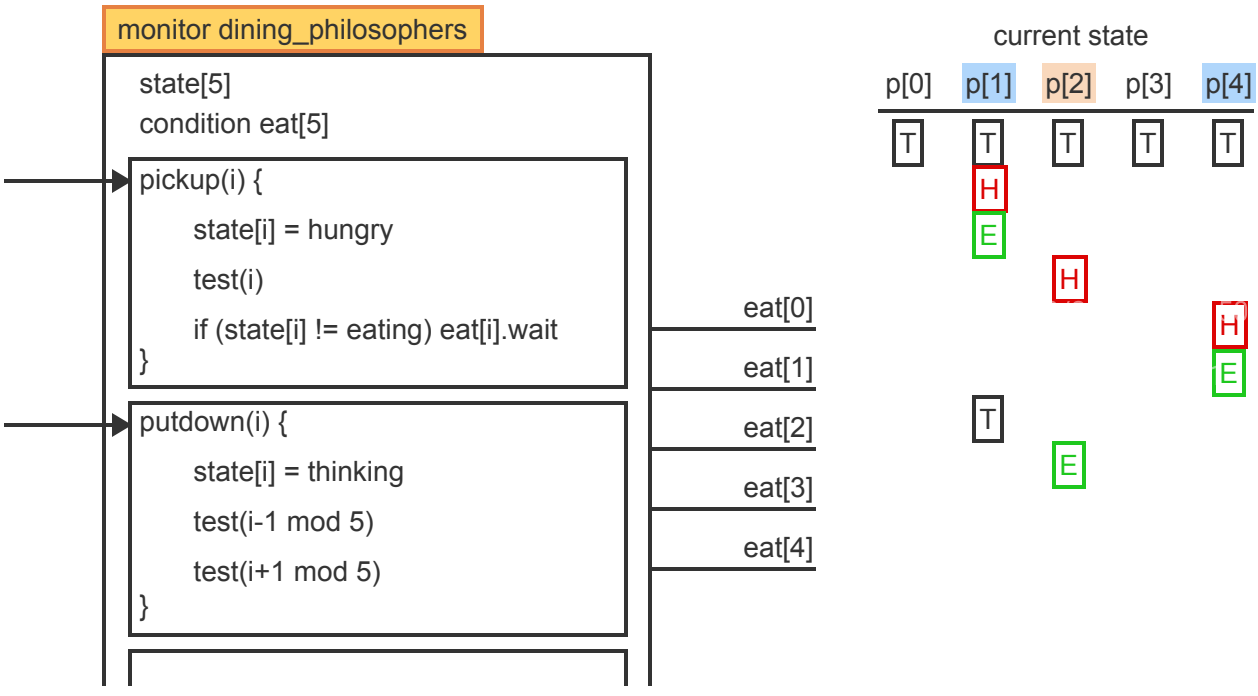
- p[1]
- p[2]
- either p[1] or p[2]

A monitor solution to the dining philosophers problem

Each of the 5 philosophers, p[i], can be in one of 3 states: thinking, hungry, or eating. Thinking does not require any forks. Hungry is a state where the philosopher is blocked because one or both forks are not available. In the eating state the philosopher is holding both forks and is executing.

- The forks are not represented explicitly by any data structure. Rather, the pickup(i) function switches p[i] from thinking to eating if p[i-1] and p[i+1] are not eating. Consequently, deadlock is not possible because no forks are picked up one at a time. Instead, the act of picking up the forks is represented implicitly by the single state transition from thinking to eating.
- Concurrent execution is guaranteed in that the putdown(i) function explicitly enables neighbor p[i-1] if p[i-2] is not eating and similarly the function enables neighbor p[i+1] if p[i+2] is not eating.

PARTICIPATION ACTIVITY 5.16.12: Behavior of the dining-philosophers monitor.



```
test(i) {
    if ((state[i-1 mod 5] != eating) &&
        (state[i] == hungry) &&
        (state[i+1 mod 5] != eating))
            state[i] = eating
            eat[i].signal
}
```

Animation content:

Static figure: A pseudocode for the monitor solution to the dining philosophers problem. monitor dining_philosophers contains three functions: pickup(), putdown(), and test(). Each function operates on an array called state[5] and there is a condition eat[5] array that corresponds to 5 queues for philosophers to wait. The current state of 5 philosophers labeled p[0] through p[4] is in a table, each cell representing the state of a philosopher with letters T for thinking, H for hungry, and E for eating.
Step 1: Five processes p[i] share the monitor. The initial state of all p[i]s is thinking (T).
Step 2: p[1] wishes to eat and calls pickup(). p[1]'s state changes to hungry (H). The internal function test() checks if the left neighbor p[i-1 mod 5] or the right neighbor p[i+1 mod 5] is eating.
Step 3: If neither neighbor is eating, p[1]'s state changes to eating (E) and p[1] leaves the monitor holding both forks. The signal in test has no effect in this case.
Step 4: While p[1] is eating, p[2] requests forks. p[2]'s state becomes H but test does not change the state to E since p[1] is eating. Thus p[2] becomes blocked on eat[2].
Step 5: If p[4] now calls pickup, the state first changes to H and then test changes the state to E since neither p[3] nor p[0] are eating. p[4] leaves the monitor holding both forks and eats concurrently with p[1].
Step 6: When p[1] finishes eating and calls putdown to return the forks, the state of p[1] changes back to T. The putdown function then calls test on both of p[1]'s neighbors, p[0] and p[2].
Step 7: When test is called on the left neighbor p[0], the state does not change to E since p[0] is not hungry.
Step 8: But when test is called on the right neighbor p[2], all three conditions are true (p[2] is hungry and neither neighbor is eating) and so the state of p[2] changes to E.

Step 9: The signal (the last instruction of test) reactivates p[2], which leaves the monitor and starts eating concurrently with p[4].

Animation captions:

1. Five processes p[i] share the monitor. The initial state of all p[i]s is thinking (T).
2. p[1] wishes to eat and calls pickup(). p[1]'s state changes to hungry (H). The internal function test() checks if the left neighbor p[i-1 mod 5] or the right neighbor p[i+1 mod 5] is eating.

3. If neither neighbor is eating, p[1]'s state changes to eating (E) and p[1] leaves the monitor holding both forks. The signal in test has no effect in this case.
4. While p[1] is eating, p[2] requests forks. p[2]'s state becomes H but test does not change the state to E since p[1] is eating. Thus p[2] becomes blocked on eat[2].
5. If p[4] now calls pickup, the state first changes to H and then test changes the state to E since neither p[3] nor p[0] are eating. p[4] leaves the monitor holding both forks and eats concurrently with p[1].
6. When p[1] finishes eating and calls putdown to return the forks, the state of p[1] changes back to T. The putdown function then calls test on both of p[1]'s neighbors, p[0] and p[2].
7. When test is called on the left neighbor p[0], the state does not change to E since p[0] is not hungry.
8. But when test is called on the right neighbor p[2], all three conditions are true and the state of p[2] changes to E.
9. The signal reactivates p[2], which leaves the monitor and starts eating concurrently with p[4].

PARTICIPATION ACTIVITY 5.16.13: Compatible states.

When state[0] == E and state[2] == H then

1) state[1] can be

- T or H or E
- only T or H
- only T

2) state[4] can be

- T or H or E
- only T or H
- only T

PARTICIPATION ACTIVITY 5.16.14: Conditions checked in the test() function.

The function test() has 3 conditions connected by logical ANDs (&&). One of the 3 conditions is always true, depending on where the function is called.

1) The condition (state[i] == hungry) is always true when p[i] calls test

- in the putdown() function on the left neighbor: test(i-1 mod 5)
- in the pickup() function on itself: test(i)
- in the putdown() function on the right neighbor: test(i+1 mod 5)

2) The condition (state[i-1 mod 5] != eating) is always true when p[i] calls test

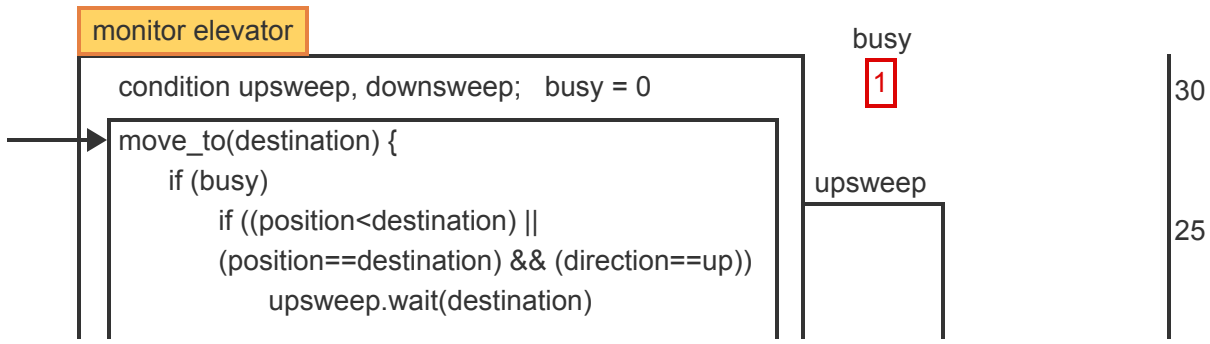
- in the putdown() function on the left neighbor: test(i-1 mod 5)
- in the pickup() function on itself: test(i)
- in the putdown() function on the right neighbor: test(i+1 mod 5)

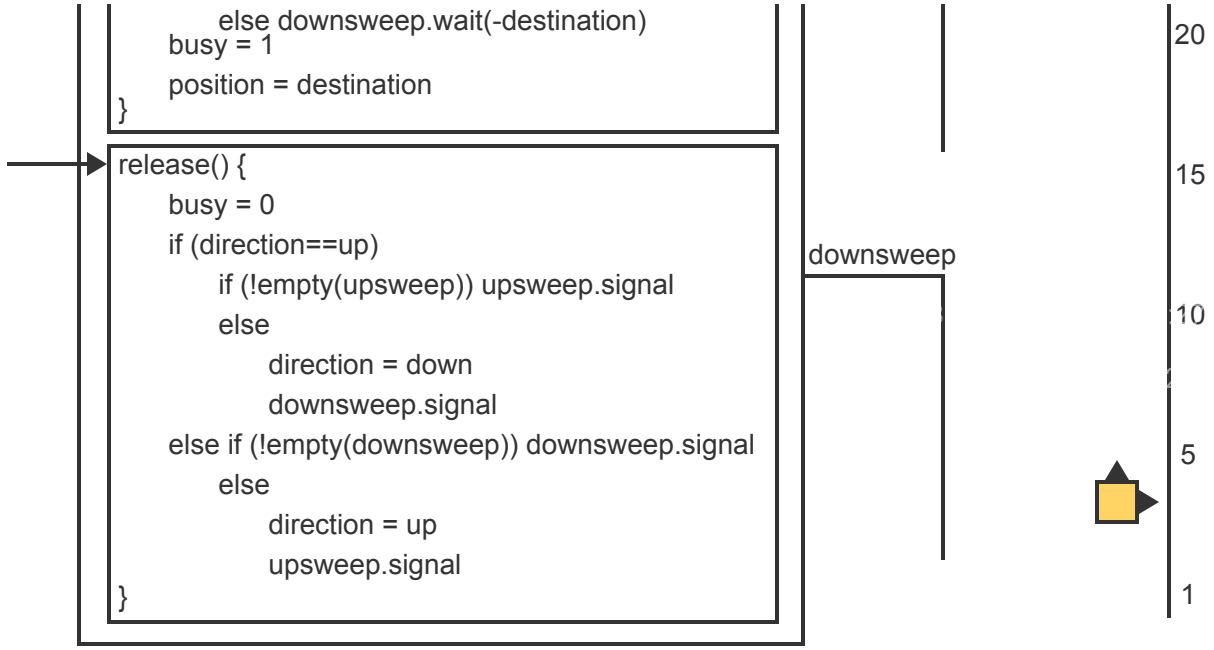
The elevator algorithm

A storage disk consists of n concentric tracks that need to be accessed by read/write requests arriving in some unspecified order. The goal is to minimize the travel distance between tracks while preventing starvation.

The elevator algorithm mimics the behavior of an elevator in an n-story building. The elevator (representing the read-write head of the disk), maintains a direction of motion, up or down, for as long as requests for floors (representing disk tracks) exist, in the current direction. When moving up and no more requests for higher floors exist, the elevator reverses direction and services all requests in descending order. When the request for the lowest floor is serviced, the elevator again reverses direction and proceeds moving up.

PARTICIPATION ACTIVITY 5.16.15: Operation of the elevator algorithm.





Animation content:

Static figure: Pseudocode of a monitor to solve the elevator problem containing 2 functions: move_to(destination) and release(). Two queues, upsweep and downsweep, allow processes to step out of the monitor and wait. A busy flag indicates whether the elevator is moving or standing at some floor. A variable “position” records the current floor. The pseudocode for move_to(destination) is:

```
if (busy)
if ((position&amp;lt;destination) ||
(position==destination) && (direction==up))
upsweep.wait(destination)
else downsweep.wait(-destination)
busy = 1
position = destination

The pseudocode for release() is:

busy = 0
if (direction==up)
if (!empty(upsweep)) upsweep.signal
else
direction = down
downsweep.signal
else if (!empty(downsweep)) downsweep.signal
else
direction = up
Upsweep.signal
```

To the right of the code snippet, a vertical line shows the floor numbers ranging from 1 to 30. The elevator is represented by a small square pointing to the current floor and showing the current direction (up or down).

Step 1: Initially, the elevator is at floor 1, the direction is up, and the elevator is not busy.
Upsweep and downsweep queues are empty, and the busy indicator is set to 0.

Step 2: A request move_to(15) arrives. The elevator is not busy so the request is not blocked. Busy is set to 1 and position is set to 15.
The elevator box moves up to floor 15.

Step 3: While the elevator is busy at floor 15, a requests move_to(25) arrives. Since 25 is above the current position 15, the request is blocked in upsweep.
A small box representing request 25 is added to the upsweep queue.

Step 4: Subsequent requests for floors 30 and 20 are also blocked in upsweep but the use of priority waits keeps all request sorted in ascending order by destination.
30 and 20 are added to the upsweep queue and all requests are sorted in ascending order: 20, 25, 30.

Step 5: Requests for floors 5,13, and 3 are inserted into downsweep but sorted from largest to smallest since the wait uses -destination as the priority.
13, 5, and 3 appear in the downsweep queue, sorted from largest to smallest.

Step 6: If the elevator is busy at floor 15 and a request for the same floor arrives then the request must be blocked according to the current direction to be serviced during the same sweep.
15 is inserted into the upsweep queue above the previous requests 20, 25, 30.

Step 7: When the first request is serviced, the process calls release. Since the elevator is still moving up and the upsweep queue is not empty, a signal reactivates the next process, which uses the same floor 15. An arrow points from upsweep.signal in release() to request 15 in upsweep queue. The elevator remains on floor 15 and request 15 is removed from the queue.

Step 8: When floor 15 is serviced, each subsequent call to release reactivates the next request and the elevator keeps moving up. When floor 30 is reached, the direction is reversed.
The elevator ascends, servicing floor 20, then 25, and finally 30. Upon reaching floor 30, the direction of the elevator switches to down.

Step 9: The downsweep.signal in release() reactivates the first request (13) from the downsweep queue and the elevator moves to the new position 13.
Request 13 is removed from the downsweep queue.

Step 10: The elevator then continues moving down until the lowest floor (3) is serviced and the direction is reversed again.

The elevator descends further, stopping at floors 5 and then floor 3, servicing each request and reversing the direction at the lowest floor.

Animation captions:

1. Initially, the elevator is at floor 1, the direction is up, and the elevator is not busy.
2. A request move_to(15) arrives. The elevator is not busy so the request is not blocked. Busy is set to 1 and position is set to 15.
3. While the elevator is busy at floor 15, a requests move_to(25) arrives. Since 25 is above the current position 15, the request is blocked in upsweep.
4. Subsequent requests for floors 30 and 20 are also blocked in upsweep but the use of priority waits keeps all request sorted in ascending order by destination.
5. Requests for floors 5,13, and 3 are inserted into downsweep but sorted from largest to smallest since the wait uses -destination as the priority.
6. If the elevator is busy at floor 15 and a request for the same floor arrives then the request must be blocked according to the current direction to be serviced during the same sweep.
7. When the first request is serviced, the process calls release. Since the elevator is still moving up and the upsweep queue is not empty, a signal reactivates the next process, which uses the same floor 15.
8. When floor 15 is serviced, each subsequent call to release reactivates the next request and the elevator keeps moving up. When floor 30 is reached, the direction is reversed.
9. The downsweep signal reactivates the first request from the downsweep queue and the elevator moves to the new position 13.
10. The elevator then continues moving down until the lowest floor (3) is serviced and the direction is reversed again.

PARTICIPATION ACTIVITY 5.16.16: Choosing the correct condition queue.

The elevator is at floor 20 and the direction is down.

1) move_to(1) will be inserted into _____

- upsweep
- downsweep

2) move_to(22) will be inserted into _____

- upsweep
- downsweep

3) move_to(20) will be inserted into _____

- upsweep
- downsweep

PARTICIPATION ACTIVITY | 5.16.17: Order of requests on condition queues.

1) The upsweep queue can contain the requests for the floors _____ in the given order.
   - ○ 15, 20, 20
   - ○ 20, 15, 20
   - ○ 20, 20, 15

2) The downsweep queue can contain the requests for the floors _____ in the given order.
   - ○ 15, 20, 20
   - ○ 20, 15, 20
   - ○ 20, 20, 15

CHALLENGE ACTIVITY | 5.16.1: The readers-writers problem.

726670.4301284.qx3zqy7

Start

Consider the monitor solution to the reader-writers problem. The first 3 columns of the table show the current contents of the ok_to_read queue, the ok_to_write queue, and the critical section CS. The next column shows new arrivals (readers or writers) at the monitor.

Given the information in the table, determine the location of each new arrival.

	ok_to_read	ok_to_write	CS	New arrivals	Location
1.	empty	empty	w1	r1	Select
2.	empty	empty	r1, r2	r3	Select
3.	empty	empty	r1, r2, r3, r4	w1	Select
4.	empty	empty	empty	r1	Select
5.	empty	empty	w1	w2	Select
6.	empty	empty	empty	w1	Select

EXERCISE 5.16.1: Order of processing readers and writers.

For the monitor solution to the readers/writers problem, assume the first request is a writer w1. While w1 is writing, the following requests all arrive in the given order:
w2, r1, r2, w3, r3, w4

(a) In which order will the requests be processed? Which groups of readers will be reading concurrently?

(b) Two new readers r4 and r5 arrive while w2 is writing. When will each request be processed?

EXERCISE 5.16.2: Simplified reader/writers problem.

(a) For the monitor solution to the readers/writers problem, assume that only one writer processes exists. Thus start_write will never be invoked until the preceding end_write has terminated. Simplify the code accordingly.

EXERCISE 5.16.3: A P/V solution to the reader/writers problem.

```
reader() {
   P(mutex)
   reading++
   if(reading == 1) P(w)
   V(mutex)
   READ
   P(mutex)
   reading--
   if(reading == 0) V(w)
   V(mutex)
}
```

```
writer() {
   P(w)
   WRITE
   V(w)
}
```

```
Initially: mutex = w = 1; reading = 0
```

(a) Does the solution satisfy the basic requirements of the readers/writers problem? Is starvation of readers or writers possible?

EXERCISE 5.16.4: A single-lane tunnel problem.

A classical synchronization problem is an analogy of a single-lane tunnel, also known as the single-lane east-west bridge. To avoid a deadlock, cars must be prevented from entering the tunnel at both ends simultaneously. Once a car enters, other cars from the same direction may follow immediately.

(a) Ignoring the problem of starvation, solve the problem using semaphores.
Hint: The tunnel problem is a variation of the readers/writers problem where multiple readers or multiple writers are allowed to enter the critical region.

EXERCISE 5.16.5: The tunnel problem without starvation.

(a) Write a monitor for the tunnel problem that guarantees that no direction of travel can be blocked indefinitely and thus prevents starvation.

EXERCISE 5.16.6: An inadequate solution for dining philosophers.

Each of the five philosophers, p[i], in the dining philosophers problem executes the code:

```
while(1){
   think
   P(mutex)
      P(f[i])
      P(f[i+1 mod 5])
   V(mutex)
   eat
   V(f[i])
   V(f[i+1 mod 5])
}
```

(a) Does the solution satisfy all requirements of the dining philosophers problem?

EXERCISE 5.16.7: Starvation in the dining philosophers problem.

The monitor solution to the dining philosophers problem can lead to starvation.

(a) Construct a scenario where 2 philosophers can repeatedly eat while the philosopher between the two starves.

EXERCISE | 5.16.8: Modified elevator solution.

The Elevator algorithm uses a complex condition to decide whether a request should be placed into the upsweep or the downsweep queue:

```
if ((position < dest) || ((position == dest) && (direction == up)))
```

(a) How will the behavior of the algorithm change if the compound condition is replaced by the simple condition:

```
if (position < dest)
```

(b) How will the behavior of the algorithm change if the compound condition is replaced by the simple condition:

```
if (position <= dest)
```